



THE UNIVERSITY OF
SYDNEY

Seat Number _____

Student Number

--	--	--	--	--	--	--	--	--	--

Family Name _____

First Name _____

CONFIDENTIAL EXAM PAPER

This paper is not to be removed from the exam venue

Electrical and Information Engineering

EXAMINATION

Semester 1 - Main, 2017

ELEC3607 Embedded Systems

EXAM WRITING TIME: 2 hours

READING TIME: 10 minutes

EXAM CONDITIONS:

This is a closed book examination - no material permitted

During reading time - writing is not permitted at all

MATERIALS PERMITTED IN THE EXAM VENUE:

(No electronic aids are permitted e.g. laptops, phones)

Calculator - non-programmable

MATERIALS TO BE SUPPLIED TO STUDENTS:

1 x 12-page answer book

INSTRUCTIONS TO STUDENTS:

1. Please check your examination paper is complete (page numbers in footer of page) and indicate by signing below.
2. Please read each question carefully. The value of each question has been written next to the question. Note that questions are of unequal value. Total mark of this exam paper is 100.
3. This examination consists of four (4) SECTIONS (A, B, C, and D).
4. ALL sections must be answered in the answer booklet provided.
5. Parts of the SAM3X datasheet and an ARM instruction set summary are provided at the end of this paper.
6. Please ensure that your Name and Student No are written on each booklet you use.

I have checked the examination paper and affirm it is complete.

Student Signature: _____ *Date:* _____

For Examiner Use Only

Q	Mark
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	
16	
17	
18	

Total _____

Section A - Peripherals (25 marks)

A data sheet for the SAM3X Analog-to-Digital Converter (ADC) peripheral is included at the end of this paper. In answering this question, you must give all constants for register addresses and values in hexadecimal.

1. (10 marks) Assuming that the chip has just been reset and is configured to have a MCK of 32 MHz (i.e., 32000000 Hz), explain the sequence required to initialize ADC. Set sampling resolution to 10-bits, ADC clock frequency to 100 KHz, startup up time to 960 μ s, analog settling time to 90 μ s, tracking time to 100 μ s, and transfer period to 70 μ s. Enable channels AD0 and AD1 for reading in their numeric order. Enable gain amplifier for all channels, and set gain to 4x.
Do not change any other options from their default nor enable anything else (no external trigger, custom order sequence, sleep mode, Power Management Controller (PMC), Peripheral DMA Controller (PDC), etc.).
2. (10 marks) In the C programming language, write a code sequence which in an infinite loop, samples channels AD0 and AD1 using polling, and prints their mean (i.e., print value of $(AD0 + AD1)/2$). Your code must read both channels after a single software trigger.
Assume every peripheral is initialized, and ADC is already configured according to your answer in part 1. Do not enable any other mode or option (for example, last transfer signals or shutdown). Your answer should not use any library calls or include files except for `Serial.println`, must be clearly commented, and use meaningful variable names. You have to redefine all registers and flags using CMSIS naming scheme.
3. (5 marks) Assume the settings used in question 1, and that ADVREF is connected to 1.024 V. If a voltage source of 128 mV is connected to port PA22, and a voltage source of 320 mV connected to PB17, what ADC channels have to be read, and what will be the conversion results for each?

Section B - ARM Assembly Language (25 marks)

An ARM instruction set summary is provided at the end of this paper.

1. (5 marks) Consider the following assembly instruction:

STMFD r13!, {r5-6}

Before executing this instruction, registers hold the following values:

Register	Value	Register	Value
r4	0x00000000	r9	0x00400040
r5	0x11223344	r10	0x00800080
r6	0x55667788	r11	0x10001000
r7	0x99aabbcc	r12	0x20002000
r8	0xddeeff00	r13	0x40004000

What memory locations are affected after executing the above instruction? In a table, with a “memory location” column and a “stored data” column, show the byte by byte values stored at each affected memory location (pay careful attention to data endianness).

Additionally, what is the effect of this instruction on registers? Only write the name and new value of the registers that change.

2. (10 marks) The following is the assembly language translation of a C function.

```
mystery:
    cmp     r1, #0
    bne     .L8
    mvn     r0, #0
    bx      lr
.L8:
    mov     r2, #0
    ldr     r12, [r0]
    mov     r3, r2
.L3:
    ldr     r2, [r0, r2, asl #2]
    add     r3, r3, #1
    cmp     r12, r2
    movcc   r12, r2
    cmp     r3, r1
    mov     r2, r3
    bne     .L3
    mov     r0, r12
    bx      lr
```

Annotate each line of the function with comments to explain what each line does. Write an equivalent C function and explain what it does. No `gotos` are allowed in C code.

3. (5 marks) Write a single instruction that would subtract 5 from r1 (i.e., in C, `r1-=5`), only if the result of last comparison instruction was an unsigned “greater than or equal”. This instruction must not change the flags.
4. (5 marks) Suppose that `r1=0xffffffff` and `r2=0x2`. After an “`add r1,r2`” instruction in Thumb-2 mode, what will be the values of r2, the negative (N) flag and the overflow (V) flag?

Section C - Interrupts (25 marks)

1. (10 marks) Assuming that the chip has just been reset, explain how to program the registers of ADC to run in Free Run mode. Enable channel AD2, and enable compare mode for it, such that a comparison is performed against 0x0180 and 0x05FF; the AD2 value is outside this window, an interrupt is raised instantly.
Assume a MCK of 32 MHz (i.e., 32000000 Hz), and set sampling resolution to 12-bits, ADC Clock frequency to 100 KHz, Startup up time to 960 μ s, Analog settling time to 90 μ s, Tracking time to 100 μ s, and Transfer period to 70 μ s. Do not change other options from their default nor enable anything else (sleep mode, Power Management Controller (PMC), Peripheral DMA Controller (PDC), etc.).
2. (10 marks) In the C programming language, write an interrupt handler `adc_irq()` that is called when an ADC Comparison event occurs. If AD2's new data is above 0x05FF, set digital pin 2 to high, and otherwise low. If the new data is below 0x0180, set digital pin 3 to high, and otherwise to low. Your answer should not use any library calls or include files (except for `digitalWrite(pin, value)`), must be clearly commented and use CMSIS naming scheme for all registers and flags. You can assume other parts of the system, such as NVIC and PIO, have been already programmed, and ADC has been programmed using your answer to part 1.
3. (5 marks) Suppose that the interrupt handler of question 2 is mission critical and must be run without delay, but there are several sources of interrupts that can delay calls to `adc_irq()`. Describe two techniques that could be used to minimize this issue.

Section D - Hardware Design (25 marks)

1. (5 marks) A Pierce crystal oscillator has been designed and built, but it is not starting. Describe 3 possible design issues that may be causing the circuit not to oscillate.
2. (5 marks) With the aid of a figure, explain ground bounce and give two techniques which can minimize this effect.

3. (5 marks) What is Manchester coding and explain a situation where it might be used.
4. (5 marks) Care needs to be taken when driving an inductive load with a digital output. Using a diagram, explain the problem and how it can be addressed.
5. (5 marks) What are the advantages of a 4-terminal linear regulator over a 3-terminal one? Explain how the pins are used differently and why it performs better.

END OF EXAMINATION

43. Analog-to-Digital Converter (ADC)

43.1 Description

The ADC is based on a 12-bit Analog-to-Digital Converter (ADC) managed by an ADC Controller. Refer to the Block Diagram: [Figure 43-1](#). It also integrates a 16-to-1 analog multiplexer, making possible the analog-to-digital conversions of 16 analog lines. The conversions extend from 0V to ADVREF. The ADC supports an 10-bit or 12-bit resolution mode, and conversion results are reported in a common register for all channels, as well as in a channel-dedicated register. Software trigger, external trigger on rising edge of the ADTRG pin or internal triggers from Timer Counter output(s) are configurable.

The comparison circuitry allows automatic detection of values below a threshold, higher than a threshold, in a given range or outside the range, thresholds and ranges being fully configurable.

The ADC Controller internal fault output is directly connected to PWM Fault input. This input can be asserted by means of comparison circuitry in order to immediately put the PWM outputs in a safe state (pure combinational path).

The ADC also integrates a Sleep Mode and a conversion sequencer and connects with a PDC channel. These features reduce both power consumption and processor intervention.

This ADC has a selectable single-ended or fully differential input and benefits from a 2-bit programmable gain. A whole set of reference voltages is generated internally from a single external reference voltage node that may be equal to the analog supply voltage. An external decoupling capacitance is required for noise filtering.

A digital error correction circuit based on the multi-bit redundant signed digit (RSD) algorithm is employed in order to reduce INL and DNL errors.

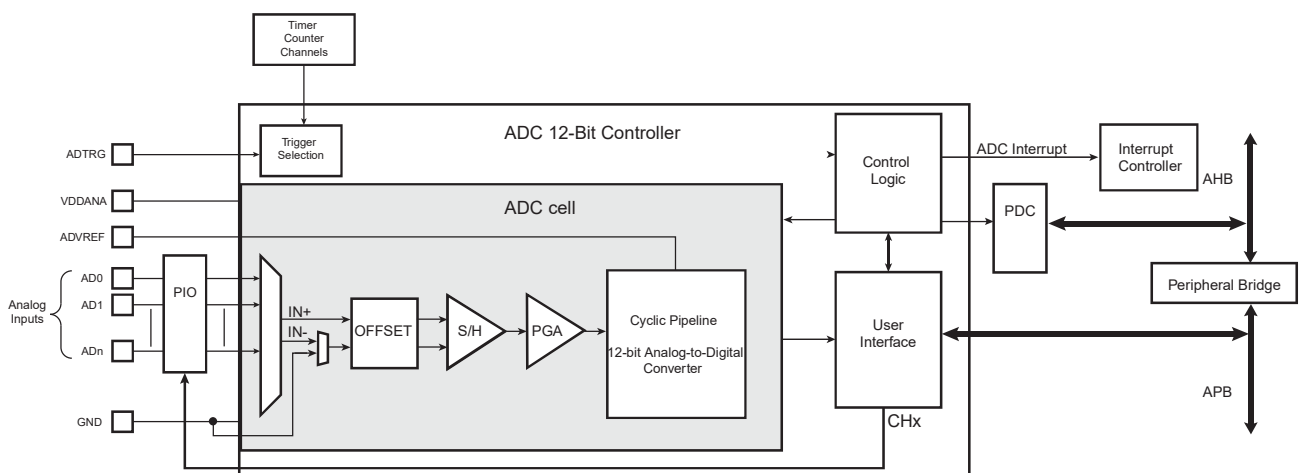
Finally, the user can configure ADC timings, such as Startup Time and Tracking Time.

43.2 Embedded Characteristics

- 12-bit Resolution
- 1 MHz Conversion Rate
- Wide Range Power Supply Operation
- Selectable Single Ended or Differential Input Voltage
- Programmable Gain For Maximum Full Scale Input Range 0 - VDD
- Integrated Multiplexer Offering Up to 16 Independent Analog Inputs
- Individual Enable and Disable of Each Channel
- Hardware or Software Trigger
 - External Trigger Pin
 - Timer Counter Outputs (Corresponding TIOA Trigger)
 - PWM Event Line
- Drive of PWM Fault Input
- PDC Support
- Possibility of ADC Timings Configuration
- Two Sleep Modes and Conversion Sequencer
 - Automatic Wakeup on Trigger and Back to Sleep Mode after Conversions of all Enabled Channels
 - Possibility of Customized Channel Sequence
- Standby Mode for Fast Wakeup Time Response
 - Power Down Capability
- Automatic Window Comparison of Converted Values
- Write Protect Registers

43.3 Block Diagram

Figure 43-1. Analog-to-Digital Converter Block Diagram



43.4 Signal Description

Table 43-1. ADC Pin Description

Pin Name	Description
VDDANA	Analog power supply
ADVREF	Reference voltage ⁽¹⁾
AD0 - AD15	Analog input channels
ADTRG	External trigger

Note: 1. AD15 is not an actual pin but is connected to a temperature sensor.

43.5 Product Dependencies

43.5.1 Power Management

The ADC Controller is not continuously clocked. The programmer must first enable the ADC Controller MCK in the Power Management Controller (PMC) before using the ADC Controller. However, if the application does not require ADC operations, the ADC Controller clock can be stopped when not needed and restarted when necessary. Configuring the ADC Controller does not require the ADC Controller clock to be enabled.

43.5.2 Interrupt Sources

The ADC interrupt line is connected on one of the internal sources of the Interrupt Controller. Using the ADC interrupt requires the NVIC to be programmed first.

Table 43-2. Peripheral IDs

Instance	ID
ADC	37

43.5.3 Analog Inputs

The analog input pins can be multiplexed with PIO lines. In this case, the assignment of the ADC input is automatically done as soon as the corresponding channel is enabled by writing the register ADC_CHER. By default, after reset, the PIO line is configured as input with its pull-up enabled and the ADC input is connected to the GND.

43.5.4 Temperature Sensor

The temperature sensor is connected to Channel 15 of the ADC.

The temperature sensor provides an output voltage V_T that is proportional to absolute temperature (PTAT). To activate the temperature sensor, TSON bit (ADC_ACR) needs to be set.

43.5.5 I/O Lines

The pin ADTRG may be shared with other peripheral functions through the PIO Controller. In this case, the PIO Controller should be set accordingly to assign the pin ADTRG to the ADC function.

Table 43-3. I/O Lines

Instance	Signal	I/O Line	Peripheral
ADC	ADTRG	PA11	B
ADC	AD0	PA2	X1
ADC	AD1/WKUP1	PA3	X1
ADC	AD2	PA4	X1
ADC	AD3	PA6	X1
ADC	AD4	PA22	X1
ADC	AD5	PA23	X1
ADC	AD6	PA24	X1
ADC	AD7	PA16	X1
ADC	AD8	PB12	X1
ADC	AD9	PB13	X1
ADC	AD10	PB17	X1
ADC	AD11	PB18	X1
ADC	AD12	PB19	X1
ADC	AD13	PB20	X1
ADC	AD14/WKUP13	PB21	X1

43.5.6 Timer Triggers

Timer Counters may or may not be used as hardware triggers depending on user requirements. Thus, some or all of the timer counters may be unconnected.

43.5.7 PWM Event Line

PWM Event Lines may or may not be used as hardware triggers depending on user requirements.

43.5.8 Fault Output

The ADC Controller has the FAULT output connected to the FAULT input of PWM. Please refer to [Section 43.6.12 "Fault Output"](#) and implementation of the PWM in the product.

43.5.9 Conversion Performances

For performance and electrical characteristics of the ADC, see the product DC Characteristics section.

43.6 Functional Description

43.6.1 Analog-to-digital Conversion

The ADC uses the ADC Clock to perform conversions. Converting a single analog value to a 12-bit digital data requires Tracking Clock cycles as defined in the field TRACKTIM of the “ADC Mode Register” on page 1333 and Transfer Clock cycles as defined in the field TRANSFER of the same register. The ADC Clock frequency is selected in the PRESCAL field of the Mode Register (ADC_MR). The tracking phase starts during the conversion of the previous channel. If the tracking time is longer than the conversion time, the tracking phase is extended to the end of the previous conversion.

The ADC clock range is between $MCK/2$, if PRESCAL is 0, and $MCK/512$, if PRESCAL is set to 255 (0xFF). PRESCAL must be programmed in order to provide an ADC clock frequency according to the parameters given in the product Electrical Characteristics section.

Figure 43-2. Sequence of ADC conversions when Tracking time > Conversion time

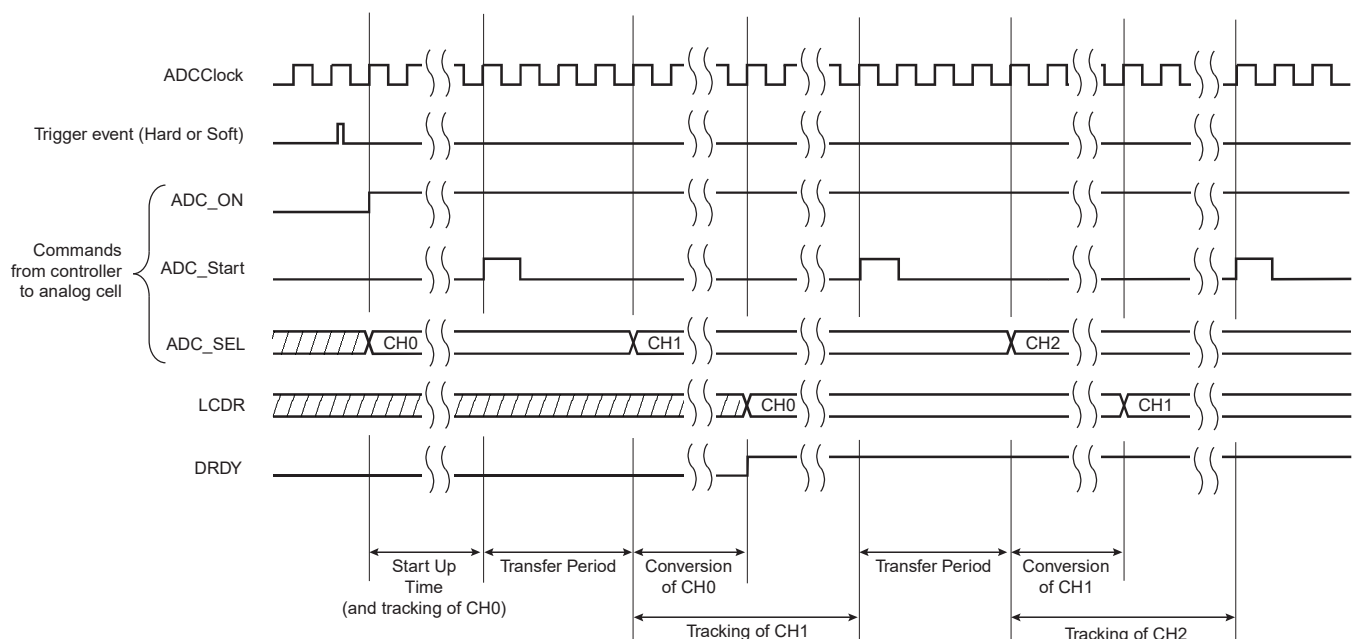
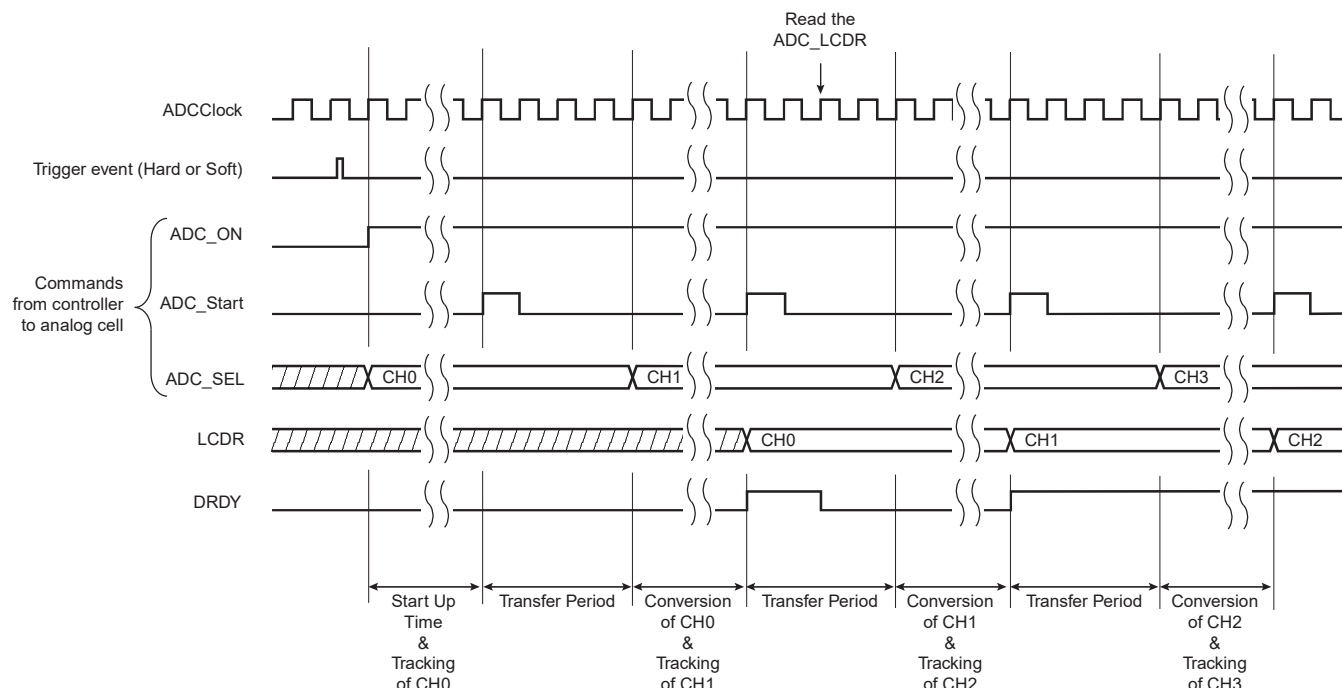


Figure 43-3. Sequence of ADC conversions when Tracking time < Conversion time



43.6.2 Conversion Reference

The conversion is performed on a full range between 0V and the reference voltage pin ADVREF. Analog inputs between these voltages convert to values based on a linear conversion.

43.6.3 Conversion Resolution

The ADC supports 10-bit or 12-bit resolutions. The 10-bit selection is performed by setting the LOWRES bit in the ADC Mode Register (ADC_MR). By default, after a reset, the resolution is the highest and the DATA field in the data registers is fully used. By setting the LOWRES bit, the ADC switches to the lowest resolution and the conversion results can be read in the lowest significant bits of the data registers. The two highest bits of the DATA field in the corresponding ADC_CDR register and of the LDATA field in the ADC_LCDR register read 0.

Moreover, when a PDC channel is connected to the ADC, 12-bit or 10-bit resolution sets the transfer request size to 16 bits.

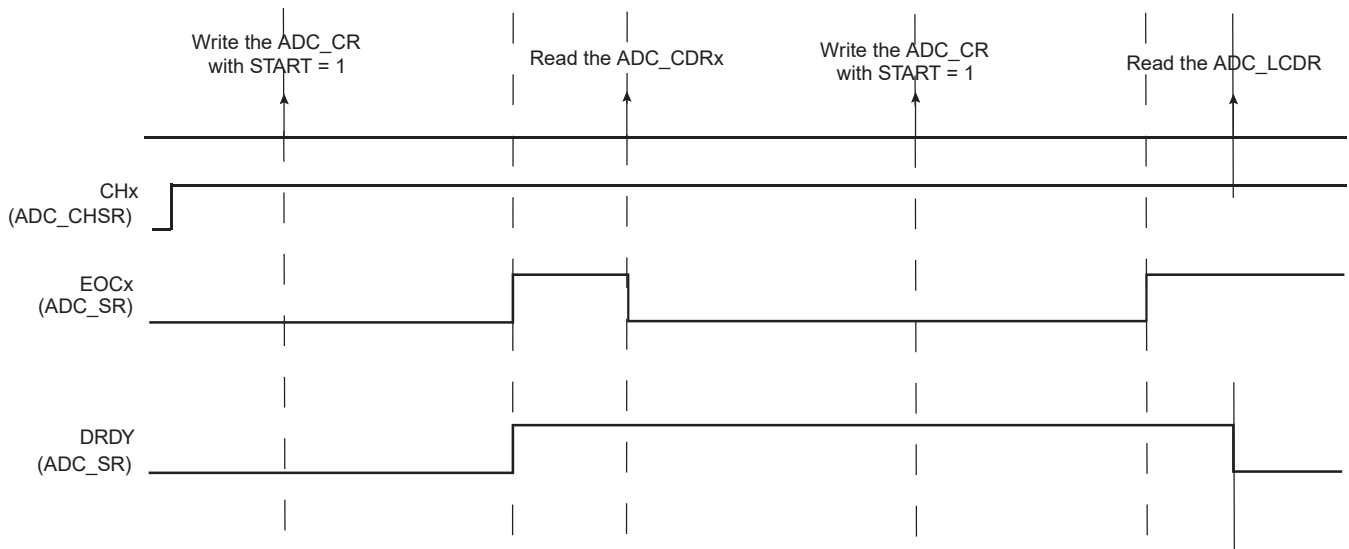
43.6.4 Conversion Results

When a conversion is completed, the resulting 12-bit digital value is stored in the Channel Data Register (ADC_CDRx) of the current channel and in the ADC Last Converted Data Register (ADC_LCDR). By setting the TAG option in the ADC_EMR, the ADC_LCDR presents the channel number associated to the last converted data in the CHNB field.

The channel EOC bit in the Status Register (ADC_SR) is set and the DRDY is set. In the case of a connected PDC channel, DRDY rising triggers a data transfer request. In any case, either EOC and DRDY can trigger an interrupt.

Reading one of the ADC_CDR registers clears the corresponding EOC bit. Reading ADC_LCDR clears the DRDY bit and EOC bit corresponding to the last converted channel.

Figure 43-4. EOCx and DRDY Flag Behavior

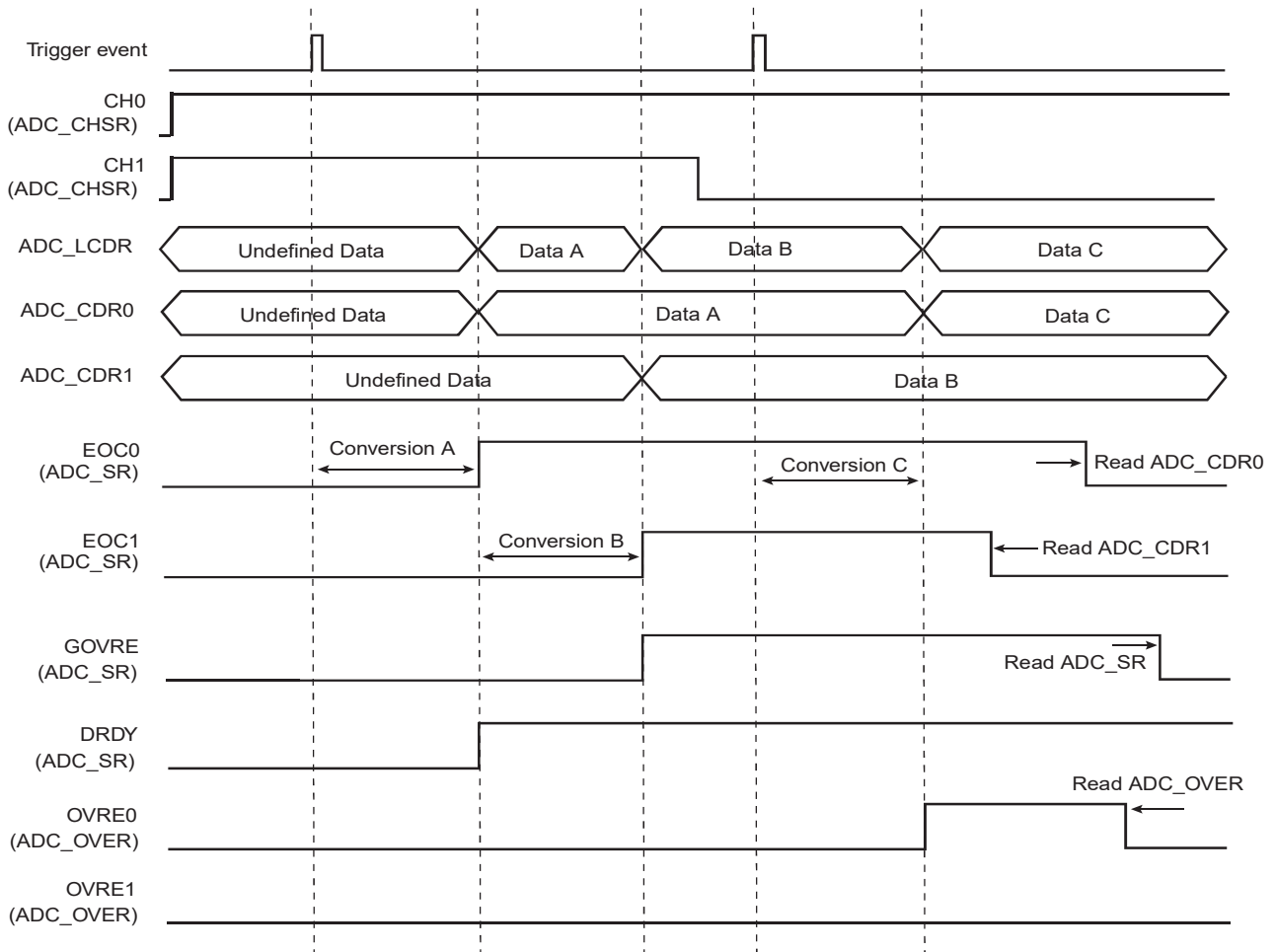


If the ADC_CDR is not read before further incoming data is converted, the corresponding Overrun Error (OVREx) flag is set in the Overrun Status Register (ADC_OVER).

Likewise, new data converted when DRDY is high sets the GOVRE bit (General Overrun Error) in ADC_SR.

The OVREx flag is automatically cleared when ADC_OVER is read, and GOVRE flag is automatically cleared when ADC_SR is read.

Figure 43-5. GOVRE and OVREx Flag Behavior



Warning: If the corresponding channel is disabled during a conversion or if it is disabled and then reenabled during a conversion, its associated data and its corresponding EOC and OVRE flags in ADC_SR are unpredictable.

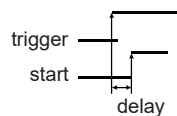
43.6.5 Conversion Triggers

Conversions of the active analog channels are started with a software or hardware trigger. The software trigger is provided by writing the Control Register (ADC_CR) with the START bit at 1.

The hardware trigger can be one of the TIOA outputs of the Timer Counter channels, PWM Event line, or the external trigger input of the ADC (ADTRG). The hardware trigger is selected with the TRGSEL field in the Mode Register (ADC_MR). The selected hardware trigger is enabled with the TRGEN bit in the Mode Register (ADC_MR).

The minimum time between 2 consecutive trigger events must be strictly greater than the duration time of the longest conversion sequence according to configuration of registers ADC_MR, ADC_CHSR, ADC_SEQR1, ADC_SEQR2.

If a hardware trigger is selected, the start of a conversion is triggered after a delay starting at each rising edge of the selected signal. Due to asynchronous handling, the delay may vary in a range of 2 MCK clock periods to 1 ADC clock period.



If one of the TIOA outputs is selected, the corresponding Timer Counter channel must be programmed in Waveform Mode.

Only one start command is necessary to initiate a conversion sequence on all the channels. The ADC hardware logic automatically performs the conversions on the active channels, then waits for a new request. The Channel Enable (ADC_CHER) and Channel Disable (ADC_CHDR) Registers permit the analog channels to be enabled or disabled independently.

If the ADC is used with a PDC, only the transfers of converted data from enabled channels are performed and the resulting data buffers should be interpreted accordingly.

43.6.6 Sleep Mode and Conversion Sequencer

The ADC Sleep Mode maximizes power saving by automatically deactivating the ADC when it is not being used for conversions. Sleep Mode is selected by setting the SLEEP bit in the Mode Register ADC_MR.

The Sleep mode is automatically managed by a conversion sequencer, which can automatically process the conversions of all channels at lowest power consumption.

This mode can be used when the minimum period of time between 2 successive trigger events is greater than the startup period of Analog-Digital converter (See the product ADC Characteristics section).

When a start conversion request occurs, the ADC is automatically activated. As the analog cell requires a start-up time, the logic waits during this time and starts the conversion on the enabled channels. When all conversions are complete, the ADC is deactivated until the next trigger. Triggers occurring during the sequence are not taken into account.

A fast wake-up mode is available in the ADC Mode Register (ADC_MR) as a compromise between power saving strategy and responsiveness. Setting the FWUP bit to '1' enables the fast wake-up mode. In fast wake-up mode the ADC cell is not fully deactivated while no conversion is requested, thereby providing less power saving but faster wakeup.

The conversion sequencer allows automatic processing with minimum processor intervention and optimized power consumption. Conversion sequences can be performed periodically using a Timer/Counter output or the PWM event line. The periodic acquisition of several samples can be processed automatically without any intervention of the processor thanks to the PDC.

The sequence can be customized by programming the Sequence Channel Registers, ADC_SEQR1 and ADC_SEQR2 and setting to 1 the USEQ bit of the Mode Register (ADC_MR). The user can choose a specific

order of channels and can program up to 16 conversions by sequence. The user is totally free to create a personal sequence, by writing channel numbers in ADC_SEQR1 and ADC_SEQR2. Not only can channel numbers be written in any sequence, channel numbers can be repeated several times. Only enabled sequence bitfields are converted, consequently to program a 15-conversion sequence, the user can simply put a disable in ADC_CHSR[15], thus disabling the 16THCH field of ADC_SEQR2.

If all ADC channels (i.e. 16) are used on an application board, there is no restriction of usage of the user sequence. But as soon as some ADC channels are not enabled for conversion but rather used as pure digital inputs, the respective indexes of these channels cannot be used in the user sequence fields (ADC_SEQR1, ADC_SEQR2 bitfields). For example, if channel 4 is disabled (ADC_CSR[4] = 0), ADC_SEQR1, ADC_SEQR2 register bitfields USCH1 up to USCH16 must not contain the value 4. Thus the length of the user sequence may be limited by this behavior.

As an example, if only 4 channels over 16 (CH0 up to CH3) are selected for ADC conversions, the user sequence length cannot exceed 4 channels. Each trigger event may launch up to 4 successive conversions of any combination of channels 0 up to 3 but no more (i.e. in this case the sequence CH0, CH0, CH1, CH1, CH1 is impossible).

A sequence that repeats several times the same channel requires more enabled channels than channels actually used for conversion. For example, a sequence like CH0, CH0, CH1, CH1 requires 4 enabled channels (4 free channels on application boards) whereas only CH0, CH1 are really converted.

Note: The reference voltage pins always remain connected in normal mode as in sleep mode.

43.6.7 Comparison Window

The ADC Controller features automatic comparison functions. It compares converted values to a low threshold or a high threshold or both, according to the CMPMODE function chosen in the Extended Mode Register (ADC_EMR). The comparison can be done on all channels or only on the channel specified in CMPSEL field of ADC_EMR. To compare all channels the CMP_ALL parameter of ADC_EMR should be set.

Moreover a filtering option can be set by writing the number of consecutive comparison errors needed to raise the flag. This number can be written and read in the CMPFILTER field of ADC_EMR.

The flag can be read on the COMPE bit of the Interrupt Status Register (ADC_ISR) and can trigger an interrupt.

The High Threshold and the Low Threshold can be read/write in the Comparison Window Register (ADC_CWR).

43.6.8 Differential Inputs

The ADC can be used either as a single ended ADC (DIFF bit equal to 0) or as a fully differential ADC (DIFF bit equal to 1) as shown in [Figure 43-6](#). By default, after a reset, the ADC is in single ended mode.

If ANACH is set in ADC_MR the ADC can apply a different mode on each channel. Otherwise the parameters of CH0 are applied to all channels.

The same inputs are used in single ended or differential mode.

In single ended mode, inputs are managed by a 16:1 channels analog multiplexer. In the fully differential mode, inputs are managed by an 8:1 channels analog multiplexer. See [Table 43-4](#) and [Table 43-5](#).

Table 43-4. Input Pins and Channel Number in Single Ended Mode

Input Pins	Channel Number
AD0	CH0
AD1	CH1
AD2	CH2
AD3	CH3
AD4	CH4

Table 43-4. Input Pins and Channel Number in Single Ended Mode (Continued)

Input Pins	Channel Number
AD5	CH5
AD6	CH6
AD7	CH7
AD8	CH8
AD9	CH9
AD10	CH10
AD11	CH11
AD12	CH12
AD13	CH13
AD14	CH14
AD15	CH15

Table 43-5. Input Pins and Channel Number In Differential Mode

Input Pins	Channel Number
AD0-AD1	CH0
AD2-AD3	CH2
AD4-AD5	CH4
AD6-AD7	CH6
AD8-AD9	CH8
AD10-AD11	CH10
AD12-AD13	CH12
AD14-AD15	CH14

43.6.9 Input Gain and Offset

The ADC has a built in Programmable Gain Amplifier (PGA) and Programmable Offset.

The Programmable Gain Amplifier can be set to gains of 1/2, 1, 2 and 4. The Programmable Gain Amplifier can be used either for single ended applications or for fully differential applications.

If ANACH is set in ADC_MR the ADC can apply different gain and offset on each channel. Otherwise the parameters of CH0 are applied to all channels.

The gain is configurable through the GAIN bit of the Channel Gain Register (ADC_CGR) as shown in [Table 43-6](#).

Table 43-6. Gain of the Sample and Hold Unit: GAIN Bits and DIFF Bit.

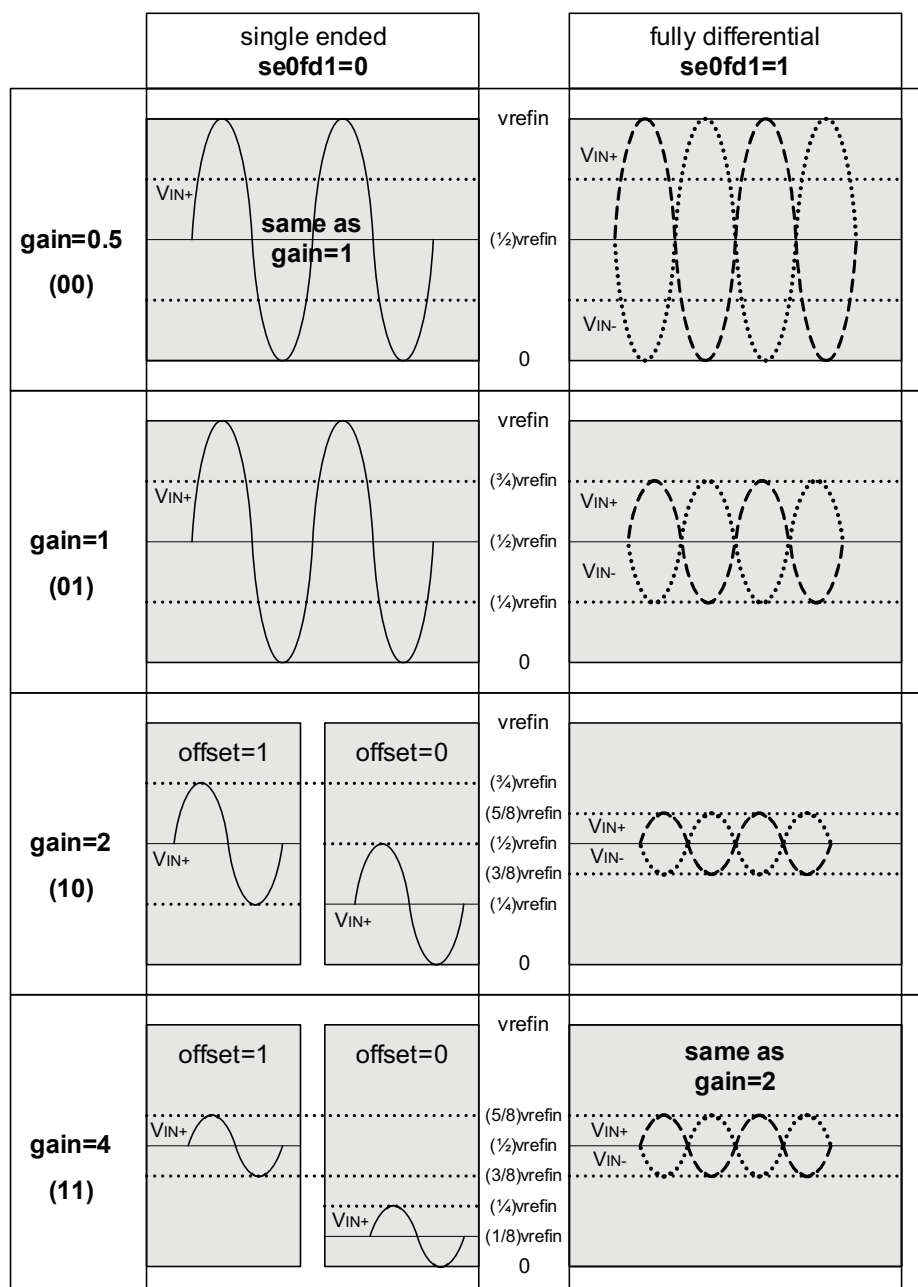
GAIN<0:1>	GAIN (DIFF = 0)	GAIN (DIFF = 1)
00	1	0.5
01	1	1
10	2	2
11	4	2

To allow full range, analog offset of the ADC can be configured by the OFFSET bit of the Channel Offset Register (ADC_COR). The Offset is only available in Single Ended Mode.

Table 43-7. Offset of the Sample and Hold Unit: OFFSET DIFF and Gain (G)

OFFSET Bit	OFFSET (DIFF = 0)	OFFSET (DIFF = 1)
0	0	0
1	$(G-1)V_{\text{refin}}/2$	

Figure 43-6. Analog Full Scale Ranges in Single Ended/Differential Applications Versus Gain and Offset



43.6.10 ADC Timings

Each ADC has its own minimal Startup Time that is programmed through the field STARTUP in the Mode Register, ADC_MR.

A minimal Tracking Time is necessary for the ADC to guarantee the best converted final value between two channel selections. This time has to be programmed through the TRACKTIM bit field in the Mode Register, ADC_MR.

When the gain, offset or differential input parameters of the analog cell change between two channels, the analog cell may need a specific settling time before starting the tracking phase. In that case, the controller automatically waits during the settling time defined in the “[ADC Mode Register](#)”. Obviously, if the ANACH option is not set, this time is unused.

Warning: No input buffer amplifier to isolate the source is included in the ADC. This must be taken into consideration to program a precise value in the TRACKTIM field. See the product ADC Characteristics section.

43.6.11 Buffer Structure

The PDC read channel is triggered each time a new data is stored in ADC_LCDR register. The same structure of data is repeatedly stored in ADC_LCDR register each time a trigger event occurs. Depending on user mode of operation (ADC_MR, ADC_CHSR, ADC_SEQR1, ADC_SEQR2) the structure differs. Each data transferred to PDC buffer, carried on a half-word (16-bit), consists of last converted data right aligned and when TAG is set in ADC_EMR register, the 4 most significant bits are carrying the channel number thus allowing an easier post-processing in the PDC buffer or better checking the PDC buffer integrity.

43.6.12 Fault Output

The ADC Controller internal fault output is directly connected to PWM fault input. Fault output may be asserted according to the configuration of ADC_EMR (Extended Mode Register) and ADC_CWR (Compare Window Register) and converted values. When the Compare occurs, the ADC fault output generates a pulse of one Master Clock Cycle to the PWM fault input. This fault line can be enabled or disabled within PWM. Should it be activated and asserted by the ADC Controller, the PWM outputs are immediately placed in a safe state (pure combinational path). Note that the ADC fault output connected to the PWM is not the COMPE bit. Thus the Fault Mode (FMODE) within the PWM configuration must be FMODE = 1.

43.6.13 Write Protection Registers

To prevent any single software error that may corrupt ADC behavior, certain address spaces can be write-protected by setting the WPEN bit in the [“ADC Write Protect Mode Register”](#) (ADC_WPMR).

If a write access to the protected registers is detected, then the WPVS flag in the ADC Write Protect Status Register (ADC_WPSR) is set and the field WPVSRC indicates in which register the write access has been attempted.

The WPVS flag is reset by writing the ADC Write Protect Mode Register (ADC_WPMR) with the appropriate access key, WPKEY.

The protected registers are:

- [“ADC Mode Register”](#) on page 1333
- [“ADC Channel Sequence 1 Register”](#) on page 1336
- [“ADC Channel Sequence 2 Register”](#) on page 1337
- [“ADC Channel Enable Register”](#) on page 1338
- [“ADC Channel Disable Register”](#) on page 1339
- [“ADC Extended Mode Register”](#) on page 1347
- [“ADC Compare Window Register”](#) on page 1348
- [“ADC Channel Gain Register”](#) on page 1349
- [“ADC Channel Offset Register”](#) on page 1350
- [“ADC Analog Control Register”](#) on page 1352

43.7 Analog-to-Digital Converter (ADC) User Interface

Any offset not listed in Table 43-8 must be considered as “reserved”.

Table 43-8. Register Mapping

Offset	Register	Name	Access	Reset
0x00	Control Register	ADC_CR	Write-only	–
0x04	Mode Register	ADC_MR	Read-write	0x00000000
0x08	Channel Sequence Register 1	ADC_SEQR1	Read-write	0x00000000
0x0C	Channel Sequence Register 2	ADC_SEQR2	Read-write	0x00000000
0x10	Channel Enable Register	ADC_CHER	Write-only	–
0x14	Channel Disable Register	ADC_CHDR	Write-only	–
0x18	Channel Status Register	ADC_CHSR	Read-only	0x00000000
0x1C	Reserved	–	–	–
0x20	Last Converted Data Register	ADC_LCDR	Read-only	0x00000000
0x24	Interrupt Enable Register	ADC_IER	Write-only	–
0x28	Interrupt Disable Register	ADC_IDR	Write-only	–
0x2C	Interrupt Mask Register	ADC_IMR	Read-only	0x00000000
0x30	Interrupt Status Register	ADC_ISR	Read-only	0x00000000
0x34	Reserved	–	–	–
0x38	Reserved	–	–	–
0x3C	Overrun Status Register	ADC_OVER	Read-only	0x00000000
0x40	Extended Mode Register	ADC_EMR	Read-write	0x00000000
0x44	Compare Window Register	ADC_CWR	Read-write	0x00000000
0x48	Channel Gain Register	ADC_CGR	Read-write	0x00000000
0x4C	Channel Offset Register	ADC_COR	Read-write	0x00000000
0x50	Channel Data Register 0	ADC_CDR0	Read-only	0x00000000
0x54	Channel Data Register 1	ADC_CDR1	Read-only	0x00000000
...	...	...	...	...
0x8C	Channel Data Register 15	ADC_CDR15	Read-only	0x00000000
0x90 - 0x90	Reserved	–	–	–
0x94	Analog Control Register	ADC_ACR	Read-write	0x00000100
0x98 - 0xAC	Reserved	–	–	–
0xC4 - 0xE0	Reserved	–	–	–
0xE4	Write Protect Mode Register	ADC_WPMR	Read-write	0x00000000
0xE8	Write Protect Status Register	ADC_WPSR	Read-only	0x00000000
0xEC - 0xF8	Reserved	–	–	–
0xFC	Reserved	–	–	–

Note: If an offset is not listed in the table it must be considered as “reserved”.

43.7.1 ADC Control Register

Name: ADC_CR

Address: 0x400C0000

Access: Write-only

31	30	29	28	27	26	25	24
–	–	–	–	–	–	–	–
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
–	–	–	–	–	–	–	–
7	6	5	4	3	2	1	0
–	–	–	–	–	–	START	SWRST

- **SWRST: Software Reset**

0 = No effect.

1 = Resets the ADC simulating a hardware reset.

- **START: Start Conversion**

0 = No effect.

1 = Begins analog-to-digital conversion.

43.7.2 ADC Mode Register

Name: ADC_MR

Address: 0x400C0004

Access: Read-write

31	30	29	28	27	26	25	24
USEQ	–	TRANSFER		TRACKTIM			
23	22	21	20	19	18	17	16
ANACH	–	SETTLING		STARTUP			
15	14	13	12	11	10	9	8
PRESCAL							
7	6	5	4	3	2	1	0
FREERUN	FWUP	SLEEP	LOWRES	TRGSEL		TRGEN	

This register can only be written if the WPEN bit is cleared in [“ADC Write Protect Mode Register” on page 1353](#).

• TRGEN: Trigger Enable

Value	Name	Description
0	DIS	Hardware triggers are disabled. Starting a conversion is only possible by software.
1	EN	Hardware trigger selected by TRGSEL field is enabled.

• TRGSEL: Trigger Selection

Value	Name	Description
0	ADC_TRIG0	External : ADCTRG
1	ADC_TRIG1	TIOA Output of the Timer Counter Channel 0
2	ADC_TRIG2	TIOA Output of the Timer Counter Channel 1
3	ADC_TRIG3	TIOA Output of the Timer Counter Channel 2
4	ADC_TRIG4	PWM Event Line 0
5	ADC_TRIG5	PWM Event Line 0
6	ADC_TRIG6	Reserved
7	–	Reserved

• LOWRES: Resolution

Value	Name	Description
0	BITS_12	12-bit resolution
1	BITS_10	10-bit resolution

• SLEEP: Sleep Mode

Value	Name	Description
0	NORMAL	Normal Mode: The ADC Core and reference voltage circuitry are kept ON between conversions
1	SLEEP	Sleep Mode: The ADC Core and reference voltage circuitry are OFF between conversions

- **FWUP: Fast Wake Up**

Value	Name	Description
0	OFF	Normal Sleep Mode: The sleep mode is defined by the SLEEP bit
1	ON	Fast Wake Up Sleep Mode: The Voltage reference is ON between conversions and ADC Core is OFF

- **FREERUN: Free Run Mode**

Value	Name	Description
0	OFF	Normal Mode
1	ON	Free Run Mode: Never wait for any trigger.

- **PRESCAL: Prescaler Rate Selection**

$$\text{ADCClock} = \text{MCK} / ((\text{PRESCAL} + 1) * 2)$$

- **STARTUP: Start Up Time**

Value	Name	Description
0	SUT0	0 periods of ADCClock
1	SUT8	8 periods of ADCClock
2	SUT16	16 periods of ADCClock
3	SUT24	24 periods of ADCClock
4	SUT64	64 periods of ADCClock
5	SUT80	80 periods of ADCClock
6	SUT96	96 periods of ADCClock
7	SUT112	112 periods of ADCClock
8	SUT512	512 periods of ADCClock
9	SUT576	576 periods of ADCClock
10	SUT640	640 periods of ADCClock
11	SUT704	704 periods of ADCClock
12	SUT768	768 periods of ADCClock
13	SUT832	832 periods of ADCClock
14	SUT896	896 periods of ADCClock
15	SUT960	960 periods of ADCClock

- **SETTLING: Analog Settling Time**

Value	Name	Description
0	AST3	3 periods of ADCClock
1	AST5	5 periods of ADCClock
2	AST9	9 periods of ADCClock
3	AST17	17 periods of ADCClock

- **ANACH: Analog Change**

Value	Name	Description
0	NONE	No analog change on channel switching: DIFF0, GAIN0 and OFF0 are used for all channels
1	ALLOWED	Allows different analog settings for each channel. See ADC_CGR and ADC_COR Registers

- **TRACKTIM: Tracking Time**

Tracking Time = (TRACKTIM + 1) * ADCClock periods.

- **TRANSFER: Transfer Period**

Transfer Period = (TRANSFER * 2 + 3) ADCClock periods.

- **USEQ: Use Sequence Enable**

Value	Name	Description
0	NUM_ORDER	Normal Mode: The controller converts channels in a simple numeric order.
1	REG_ORDER	User Sequence Mode: The sequence respects what is defined in ADC_SEQR1 and ADC_SEQR2 registers.

43.7.3 ADC Channel Sequence 1 Register

Name: ADC_SEQR1

Address: 0x400C0008

Access: Read-write

31	30	29	28	27	26	25	24
USCH8				USCH7			
23	22	21	20	19	18	17	16
USCH6				USCH5			
15	14	13	12	11	10	9	8
USCH4				USCH3			
7	6	5	4	3	2	1	0
USCH2				USCH1			

This register can only be written if the WPEN bit is cleared in [“ADC Write Protect Mode Register” on page 1353](#).

- **USCHx: User Sequence Number x**

The sequence number x (USCHx) can be programmed by the Channel number CHy where y is the value written in this field. The allowed range is 0 up to 15. So it is only possible to use the sequencer from CH0 to CH15.

This register activates only if ADC_MR(USEQ) field is set to '1'.

Any USCHx field is taken into account only if ADC_CHSR(CHx) register field reads logical '1' else any value written in USCHx does not add the corresponding channel in the conversion sequence.

Configuring the same value in different fields leads to multiple samples of the same channel during the conversion sequence. This can be done consecutively, or not, according to user needs.

43.7.4 ADC Channel Sequence 2 Register

Name: ADC_SEQR2

Address: 0x400C000C

Access: Read-write

31	30	29	28	27	26	25	24
USCH16				USCH15			
23	22	21	20	19	18	17	16
USCH14				USCH13			
15	14	13	12	11	10	9	8
USCH12				USCH11			
7	6	5	4	3	2	1	0
USCH10				USCH9			

This register can only be written if the WPEN bit is cleared in [“ADC Write Protect Mode Register” on page 1353](#).

- **USCHx: User Sequence Number x**

The sequence number x (USCHx) can be programmed by the Channel number CHy where y is the value written in this field. The allowed range is 0 up to 15. So it is only possible to use the sequencer from CH0 to CH15.

This register activates only if ADC_MR(USEQ) field is set to '1'.

Any USCHx field is taken into account only if ADC_CHSR(CHx) register field reads logical '1' else any value written in USCHx does not add the corresponding channel in the conversion sequence.

Configuring the same value in different fields leads to multiple samples of the same channel during the conversion sequence. This can be done consecutively, or not, according to user needs.

43.7.5 ADC Channel Enable Register

Name: ADC_CHER

Address: 0x400C0010

Access: Write-only

31	30	29	28	27	26	25	24
–	–	–	–	–	–	–	–
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
CH15	CH14	CH13	CH12	CH11	CH10	CH9	CH8
7	6	5	4	3	2	1	0
CH7	CH6	CH5	CH4	CH3	CH2	CH1	CH0

This register can only be written if the WPEN bit is cleared in [“ADC Write Protect Mode Register” on page 1353](#).

- **CHx: Channel x Enable**

0 = No effect.

1 = Enables the corresponding channel.

Note: if USEQ = 1 in ADC_MR register, CHx corresponds to the xth channel of the sequence described in ADC_SEQR1 and ADC_SEQR2.

43.7.6 ADC Channel Disable Register

Name: ADC_CHDR

Address: 0x400C0014

Access: Write-only

31	30	29	28	27	26	25	24
–	–	–	–	–	–	–	–
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
CH15	CH14	CH13	CH12	CH11	CH10	CH9	CH8
7	6	5	4	3	2	1	0
CH7	CH6	CH5	CH4	CH3	CH2	CH1	CH0

This register can only be written if the WPEN bit is cleared in [“ADC Write Protect Mode Register” on page 1353](#).

- **CHx: Channel x Disable**

0 = No effect.

1 = Disables the corresponding channel.

Warning: If the corresponding channel is disabled during a conversion or if it is disabled then reenabled during a conversion, its associated data and its corresponding EOC and OVRE flags in ADC_SR are unpredictable.

43.7.7 ADC Channel Status Register

Name: ADC_CHSR

Address: 0x400C0018

Access: Read-only

31	30	29	28	27	26	25	24
–	–	–	–	–	–	–	–
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
CH15	CH14	CH13	CH12	CH11	CH10	CH9	CH8
7	6	5	4	3	2	1	0
CH7	CH6	CH5	CH4	CH3	CH2	CH1	CH0

- **CHx: Channel x Status**

0 = Corresponding channel is disabled.

1 = Corresponding channel is enabled.

43.7.8 ADC Last Converted Data Register

Name: ADC_LCDR

Address: 0x400C0020

Access: Read-only

31	30	29	28	27	26	25	24
–	–	–	–	–	–	–	–
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
CHNB				LDATA			
7	6	5	4	3	2	1	0
LDATA							

- **LDATA: Last Data Converted**

The analog-to-digital conversion data is placed into this register at the end of a conversion and remains until a new conversion is completed.

- **CHNB: Channel Number**

Indicates the last converted channel when the TAG option is set to 1 in ADC_EMR register. If TAG option is not set, CHNB = 0.

43.7.9 ADC Interrupt Enable Register

Name: ADC_IER

Address: 0x400C0024

Access: Write-only

31	30	29	28	27	26	25	24
–	–	–	RXBUFF	ENDRX	COMPE	GOVRE	DRDY
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
EOC15	EOC14	EOC13	EOC12	EOC11	EOC10	EOC9	EOC8
7	6	5	4	3	2	1	0
EOC7	EOC6	EOC5	EOC4	EOC3	EOC2	EOC1	EOC0

- **EOCx:** End of Conversion Interrupt Enable x
- **DRDY:** Data Ready Interrupt Enable
- **GOVRE:** General Overrun Error Interrupt Enable
- **COMPE:** Comparison Event Interrupt Enable
- **ENDRX:** End of Receive Buffer Interrupt Enable
- **RXBUFF:** Receive Buffer Full Interrupt Enable

0 = No effect.

1 = Enables the corresponding interrupt.

43.7.10 ADC Interrupt Disable Register

Name: ADC_IDR

Address: 0x400C0028

Access: Write-only

31	30	29	28	27	26	25	24
–	–	–	RXBUFF	ENDRX	COMPE	GOVRE	DRDY
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
EOC15	EOC14	EOC13	EOC12	EOC11	EOC10	EOC9	EOC8
7	6	5	4	3	2	1	0
EOC7	EOC6	EOC5	EOC4	EOC3	EOC2	EOC1	EOC0

- **EOCx:** End of Conversion Interrupt Disable x
- **DRDY:** Data Ready Interrupt Disable
- **GOVRE:** General Overrun Error Interrupt Disable
- **COMPE:** Comparison Event Interrupt Disable
- **ENDRX:** End of Receive Buffer Interrupt Disable
- **RXBUFF:** Receive Buffer Full Interrupt Disable

0 = No effect.

1 = Disables the corresponding interrupt.

43.7.11 ADC Interrupt Mask Register

Name: ADC_IMR

Address: 0x400C002C

Access: Read-only

31	30	29	28	27	26	25	24
–	–	–	RXBUFF	ENDRX	COMPE	GOVRE	DRDY
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
EOC15	EOC14	EOC13	EOC12	EOC11	EOC10	EOC9	EOC8
7	6	5	4	3	2	1	0
EOC7	EOC6	EOC5	EOC4	EOC3	EOC2	EOC1	EOC0

- **EOCx:** End of Conversion Interrupt Mask x
- **DRDY:** Data Ready Interrupt Mask
- **GOVRE:** General Overrun Error Interrupt Mask
- **COMPE:** Comparison Event Interrupt Mask
- **ENDRX:** End of Receive Buffer Interrupt Mask
- **RXBUFF:** Receive Buffer Full Interrupt Mask

0 = The corresponding interrupt is disabled.

1 = The corresponding interrupt is enabled.

43.7.12 ADC Interrupt Status Register

Name: ADC_ISR

Address: 0x400C0030

Access: Read-only

31	30	29	28	27	26	25	24
–	–	–	RXBUFF	ENDRX	COMPE	GOVRE	DRDY
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
EOC15	EOC14	EOC13	EOC12	EOC11	EOC10	EOC9	EOC8
7	6	5	4	3	2	1	0
EOC7	EOC6	EOC5	EOC4	EOC3	EOC2	EOC1	EOC0

- **EOCx: End of Conversion x**

0 = Corresponding analog channel is disabled, or the conversion is not finished. This flag is cleared when reading the corresponding ADC_CDRx registers.

1 = Corresponding analog channel is enabled and conversion is complete.

- **DRDY: Data Ready**

0 = No data has been converted since the last read of ADC_LCDR.

1 = At least one data has been converted and is available in ADC_LCDR.

- **GOVRE: General Overrun Error**

0 = No General Overrun Error occurred since the last read of ADC_ISR.

1 = At least one General Overrun Error has occurred since the last read of ADC_ISR.

- **COMPE: Comparison Error**

0 = No Comparison Error since the last read of ADC_ISR.

1 = At least one Comparison Error has occurred since the last read of ADC_ISR.

- **ENDRX: End of RX Buffer**

0 = The Receive Counter Register has not reached 0 since the last write in ADC_RCR or ADC_RNCR.

1 = The Receive Counter Register has reached 0 since the last write in ADC_RCR or ADC_RNCR.

- **RXBUFF: RX Buffer Full**

0 = ADC_RCR or ADC_RNCR have a value other than 0.

1 = Both ADC_RCR and ADC_RNCR have a value of 0.

43.7.13 ADC Overrun Status Register

Name: ADC_OVER

Address: 0x400C003C

Access: Read-only

31	30	29	28	27	26	25	24
–	–	–	–	–	–	–	–
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
OVRE15	OVRE14	OVRE13	OVRE12	OVRE11	OVRE10	OVRE9	OVRE8
7	6	5	4	3	2	1	0
OVRE7	OVRE6	OVRE5	OVRE4	OVRE3	OVRE2	OVRE1	OVRE0

- **OVREx: Overrun Error x**

0 = No overrun error on the corresponding channel since the last read of ADC_OVER.

1 = There has been an overrun error on the corresponding channel since the last read of ADC_OVER.

43.7.14 ADC Extended Mode Register

Name: ADC_EMR

Address: 0x400C0040

Access: Read-write

31	30	29	28	27	26	25	24
–	–	–	–	–	–	–	TAG
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
–	–	CMPFILTER			–	–	CMPALL
7	6	5	4	3	2	1	0
CMPSEL				–	–	CMPMODE	

This register can only be written if the WPEN bit is cleared in [“ADC Write Protect Mode Register” on page 1353](#).

- **CMPMODE: Comparison Mode**

Value	Name	Description
0	LOW	Generates an event when the converted data is lower than the low threshold of the window.
1	HIGH	Generates an event when the converted data is higher than the high threshold of the window.
2	IN	Generates an event when the converted data is in the comparison window.
3	OUT	Generates an event when the converted data is out of the comparison window.

- **CMPSEL: Comparison Selected Channel**

If CMPALL = 0: CMPSEL indicates which channel has to be compared.

If CMPALL = 1: No effect.

- **CMPALL: Compare All Channels**

0 = Only channel indicated in CMPSEL field is compared.

1 = All channels are compared.

- **CMPFILTER: Compare Event Filtering**

Number of consecutive compare events necessary to raise the flag = CMPFILTER+1

When programmed to 0, the flag rises as soon as an event occurs.

- **TAG: TAG of ADC_LDCR register**

0 = set CHNB to zero in ADC_LDCR.

1 = append the channel number to the conversion result in ADC_LDCR register.

43.7.15 ADC Compare Window Register

Name: ADC_CWR

Address: 0x400C0044

Access: Read-write

31	30	29	28	27	26	25	24
–	–	–	–	HIGHTHRES			
23	22	21	20	19	18	17	16
HIGHTHRES							
15	14	13	12	11	10	9	8
–	–	–	–	LOWTHRES			
7	6	5	4	3	2	1	0
LOWTHRES							

This register can only be written if the WPEN bit is cleared in [“ADC Write Protect Mode Register”](#) on page 1353.

- **LOWTHRES: Low Threshold**

Low threshold associated to compare settings of ADC_EMR register.

- **HIGHTHRES: High Threshold**

High threshold associated to compare settings of ADC_EMR register.

43.7.16 ADC Channel Gain Register

Name: ADC_CGR

Address: 0x400C0048

Access: Read-write

31	30	29	28	27	26	25	24
GAIN15		GAIN14		GAIN13		GAIN12	
23	22	21	20	19	18	17	16
GAIN11		GAIN10		GAIN9		GAIN8	
15	14	13	12	11	10	9	8
GAIN7		GAIN6		GAIN5		GAIN4	
7	6	5	4	3	2	1	0
GAIN3		GAIN2		GAIN1		GAIN0	

This register can only be written if the WPEN bit is cleared in [“ADC Write Protect Mode Register” on page 1353](#).

- **GAINx: Gain for channel x**

Gain applied on input of analog-to-digital converter.

GAINx		Gain applied when DIFFx = 0	Gain applied when DIFFx = 1
0	0	1	0.5
0	1	1	1
1	0	2	2
1	1	4	2

The DIFFx mentioned in this table is described in the following register, ADC_COR.

43.7.17 ADC Channel Offset Register

Name: ADC_COR

Address: 0x400C004C

Access: Read-write

31	30	29	28	27	26	25	24
DIFF15	DIFF14	DIFF13	DIFF12	DIFF11	DIFF10	DIFF9	DIFF8
23	22	21	20	19	18	17	16
DIFF7	DIFF6	DIFF5	DIFF4	DIFF3	DIFF2	DIFF1	DIFF0
15	14	13	12	11	10	9	8
OFF15	OFF14	OFF13	OFF12	OFF11	OFF10	OFF9	OFF8
7	6	5	4	3	2	1	0
OFF7	OFF6	OFF5	OFF4	OFF3	OFF2	OFF1	OFF0

This register can only be written if the WPEN bit is cleared in [“ADC Write Protect Mode Register” on page 1353](#).

- **OFFx: Offset for channel x**

0 = No Offset.

1 = center the analog signal on Vrefin/2 before the gain scaling. The Offset applied is: $(G-1)V_{\text{refin}}/2$

where G is the gain applied (see description of ADC_CGR register).

- **DIFFx: Differential inputs for channel x**

0 = Single Ended Mode.

1 = Fully Differential Mode.

43.7.18 ADC Channel Data Register

Name: ADC_CDRx [x=0..15]

Address: 0x400C0050

Access: Read-only

31	30	29	28	27	26	25	24
–	–	–	–	–	–	–	–
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
–	–	–	–	DATA			
7	6	5	4	3	2	1	0
DATA							

- **DATA: Converted Data**

The analog-to-digital conversion data is placed into this register at the end of a conversion and remains until a new conversion is completed. The Convert Data Register (CDR) is only loaded if the corresponding analog channel is enabled.

43.7.19 ADC Analog Control Register

Name: ADC_ACR

Address: 0x400C0094

Access: Read-only

31	30	29	28	27	26	25	24
–	–	–	–	–	–	–	–
23	22	21	20	19	18	17	16
–	–	–	–	–	–	–	–
15	14	13	12	11	10	9	8
–	–	–	–	–	–	IBCTL	
7	6	5	4	3	2	1	0
–	–	–	TSON	–	–	–	–

This register can only be written if the WPEN bit is cleared in [“ADC Write Protect Mode Register” on page 1353](#).

- **TSON: Temperature Sensor On**

0 = temperature sensor is off.

1 = temperature sensor is on.

- **IBCTL: ADC Bias Current Control**

Allows to adapt performance versus power consumption. (See the product electrical characteristics for further details.)

43.7.20 ADC Write Protect Mode Register

Name: ADC_WPMR

Address: 0x400C00E4

Access: Read-write

31	30	29	28	27	26	25	24
WPKEY							
23	22	21	20	19	18	17	16
WPKEY							
15	14	13	12	11	10	9	8
WPKEY							
7	6	5	4	3	2	1	0
–	–	–	–	–	–	–	WPEN

- **WPEN: Write Protect Enable**

0 = Disables the Write Protect if WPKEY corresponds to 0x414443 (“ADC” in ASCII).

1 = Enables the Write Protect if WPKEY corresponds to 0x414443 (“ADC” in ASCII).

Protects the registers:

[“ADC Mode Register” on page 1333](#)

[“ADC Channel Sequence 1 Register” on page 1336](#)

[“ADC Channel Sequence 2 Register” on page 1337](#)

[“ADC Channel Enable Register” on page 1338](#)

[“ADC Channel Disable Register” on page 1339](#)

[“ADC Extended Mode Register” on page 1347](#)

[“ADC Compare Window Register” on page 1348](#)

[“ADC Channel Gain Register” on page 1349](#)

[“ADC Channel Offset Register” on page 1350](#)

[“ADC Analog Control Register” on page 1352](#)

- **WPKEY: Write Protect KEY**

Should be written at value 0x414443 (“ADC” in ASCII). Writing any other value in this field aborts the write operation of the WPEN bit. Always reads as 0.

43.7.21 ADC Write Protect Status Register

Name: ADC_WPSR

Address: 0x400C00E8

Access: Read-only

31	30	29	28	27	26	25	24
–	–	–	–	–	–	–	–
23	22	21	20	19	18	17	16
WPVSR							
15	14	13	12	11	10	9	8
WPVSR							
7	6	5	4	3	2	1	0
–	–	–	–	–	–	–	WPVS

- **WPVS: Write Protect Violation Status**

0 = No Write Protect Violation has occurred since the last read of the ADC_WPSR register.

1 = A Write Protect Violation has occurred since the last read of the ADC_WPSR register. If this violation is an unauthorized attempt to write a protected register, the associated violation is reported into field WPVSR.

- **WPVSR: Write Protect Violation Source**

When WPVS is active, this field indicates the write-protected register (through address offset or code) in which a write access has been attempted.

Reading ADC_WPSR automatically clears all fields.

ARM® and Thumb®-2 Instruction Set Quick Reference Card

Key to Tables		
Rm {, <opsh>}	See Table Register, optionally shifted by constant	
<Operand2>	See Table Flexible Operand 2 . Shift and rotate are only available as part of Operand2.	
<fields>	See Table PSR fields .	<reglist>
<PSR>	Either CPSR (Current Processor Status Register) or SPSR (Saved Processor Status Register)	<reglist-PC>
C*, V*	Flag is unpredictable in Architecture v4 and earlier, unchanged in Architecture v5 and later.	<reglist+PC>
<Rs sh>	Can be Rs or an immediate shift value. The values allowed for each shift type are the same as those shown in Table Register, optionally shifted by constant .	+/-
x, y	B meaning half-register [15:0], or T meaning [31:16].	\$
<imm8m>	ARM: a 32-bit constant, formed by right-rotating an 8-bit value by an even number of bits. Thumb: a 32-bit constant, formed by left-shifting an 8-bit value by any number of bits, or a bit pattern of one of the forms 0xYXYXYXY, 0x00XY00XY or 0xXY00XY00.	<iflags>
<prefix>	See Table Prefixes for Parallel instructions	<p_mode>
{ IA TB DA DB }	Increment After, Increment Before, Decrement After, or Decrement Before.	SPm
<size>	IB and DA are not available in Thumb state. If omitted, defaults to IA. B, SB, H, or SH, meaning Byte, Signed Byte, Halfword, and Signed Halfword respectively. SB and SH are not available in STR instructions.	<lsb>
		<width>
		{X}
		{I}
		{S}
		{T}
		{R}

Operation	\$	Assembler	S updates	Action	Notes
Add					
Add with carry		ADD{S} Rd, Rn, <Operand2>	N Z C V	Rd := Rn + Operand2	N
wide	T2	ADC{S} Rd, Rn, <Operand2>	N Z C V	Rd := Rn + Operand2 + Carry	N
saturation {doubled}	5E	Q{D} ADD Rd, Rm, Rn		Rd := Rn + imm12, imm12 range 0-4095	T, P
Form PC-relative address		ADR Rd, <label>		Rd := SAT(Rm + Rn)	Q
Subtract					
Subtract with carry		SUB{S} Rd, Rn, <Operand2>	N Z C V	Rd := Rn - Operand2	N, L
wide	T2	SBC{S} Rd, Rn, <Operand2>	N Z C V	Rd := Rn - Operand2 - NOT(Carry)	N
reverse subtract		SUB Rd, Rn, #<imm12>	N Z C V	Rd := Rn - imm12, imm12 range 0-4095	N
reverse subtract with carry		RSB{S} Rd, Rn, <Operand2>	N Z C V	Rd := Operand2 - Rn	T, P
saturation {doubled}	5E	RSC{S} Rd, Rn, <Operand2>	N Z C V	Rd := Operand2 - Rn	N
Exception return without stack		Q{D} SUB Rd, Rm, Rn	N Z C V	Rd := Operand2 - Rn - NOT(Carry)	A
		SUBS PC, LR, #<imm8>		Rd := SAT(Rm - Rn)	Q
				PC = LR - imm8, CPSR = SPSR(current mode), imm8 range 0-255.	T
Parallel arithmetic					
Halfword-wise addition	6	<prefix>ADD16 Rd, Rn, Rm		Rd[31:16] := Rn[31:16] + Rm[31:16], Rd[15:0] := Rn[15:0] + Rm[15:0]	G
Halfword-wise subtraction	6	<prefix>SUB16 Rd, Rn, Rm		Rd[31:16] := Rn[31:16] - Rm[31:16], Rd[15:0] := Rn[15:0] - Rm[15:0]	G
Byte-wise addition	6	<prefix>ADD8 Rd, Rn, Rm		Rd[31:24] := Rn[31:24] + Rm[31:24], Rd[23:16] := Rn[23:16] + Rm[23:16], Rd[15:8] := Rn[15:8] + Rm[15:8], Rd[7:0] := Rn[7:0] + Rm[7:0]	G
Byte-wise subtraction	6	<prefix>SUB8 Rd, Rn, Rm		Rd[31:24] := Rn[31:24] - Rm[31:24], Rd[23:16] := Rn[23:16] - Rm[23:16], Rd[15:8] := Rn[15:8] - Rm[15:8], Rd[7:0] := Rn[7:0] - Rm[7:0]	G
Halfword-wise exchange, add, subtract	6	<prefix>ASX Rd, Rn, Rm		Rd[31:16] := Rn[31:16] + Rm[15:0], Rd[15:0] := Rn[15:0] - Rm[31:16]	G
Halfword-wise exchange, subtract, add	6	<prefix>SAX Rd, Rn, Rm		Rd[31:16] := Rn[31:16] - Rm[15:0], Rd[15:0] := Rn[15:0] + Rm[31:16]	G
Unsigned sum of absolute differences	6	USAD8 Rd, Rm, Rs		Rd := Abs(Rm[31:24] - Rs[31:24]) + Abs(Rm[23:16] - Rs[23:16]) + Abs(Rm[15:8] - Rs[15:8]) + Abs(Rm[7:0] - Rs[7:0])	
and accumulate	6	USADA8 Rd, Rm, Rs, Rn		Rd := Rn + Abs(Rm[31:24] - Rs[31:24]) + Abs(Rm[23:16] - Rs[23:16]) + Abs(Rm[15:8] - Rs[15:8]) + Abs(Rm[7:0] - Rs[7:0])	
Saturate					
Signed saturate word, right shift	6	SSAT Rd, #<sat>, Rm{, ASR <sh>}		Rd := SignedSat((Rm ASR sh), sat). <sat> range 1-32, <sh> range 1-31.	Q, R
Signed saturate word, left shift	6	SSAT Rd, #<sat>, Rm{, LSL <sh>}		Rd := SignedSat((Rm LSL sh), sat). <sat> range 1-32, <sh> range 0-31.	Q
Signed saturate two halfwords	6	SSAT16 Rd, #<sat>, Rm		Rd[31:16] := SignedSat(Rm[31:16], sat), Rd[15:0] := SignedSat(Rm[15:0], sat).	Q
Unsigned saturate word, right shift	6	USAT Rd, #<sat>, Rm{, ASR <sh>}		Rd := UnsignedSat((Rm ASR sh), sat). <sat> range 1-31, <sh> range 1-16.	Q, R
Unsigned saturate word, left shift	6	USAT Rd, #<sat>, Rm{, LSL <sh>}		Rd := UnsignedSat((Rm LSL sh), sat). <sat> range 0-31, <sh> range 0-31.	Q
Unsigned saturate two halfwords	6	USAT16 Rd, #<sat>, Rm		Rd[31:16] := UnsignedSat(Rm[31:16], sat), Rd[15:0] := UnsignedSat(Rm[15:0], sat).	Q

ARM and Thumb-2 Instruction Set Quick Reference Card

Operation	§	Assembler	S updates	Action	Notes
Multiply	Multiply and accumulate and subtract	MUL{S} Rd, Rm, Rs	N Z C*	Rd := (Rm * Rs)[31:0] (If Rm is Rd, S can be used in Thumb-2)	N, S
		MLA{S} Rd, Rm, Rs, Rn	N Z C*	Rd := (Rn + (Rm * Rs))[31:0]	S
	unsigned long	MLS Rd, Rm, Rs, Rn	N Z C*	Rd := (Rn - (Rm * Rs))[31:0]	S
		UMLL{S} RdLo, RdHi, Rm, Rs	N Z C* V*	RdHi, RdLo := unsigned(Rm * Rs)	S
	unsigned accumulate long	UMLAL{S} RdLo, RdHi, Rm, Rs	N Z C* V*	RdHi, RdLo := unsigned(RdHi, RdLo + Rm * Rs)	S
		UMLAL{S} RdLo, RdHi, Rm, Rs	N Z C* V*	RdHi, RdLo := signed(RdHi, RdLo + Rm * Rs)	S
	Signed multiply long and accumulate long	SMUL{S} RdLo, RdHi, Rm, Rs	N Z C* V*	RdHi, RdLo := signed(Rm * Rs)	S
		SMUL{S} RdLo, RdHi, Rm, Rs	N Z C* V*	RdHi, RdLo := signed(RdHi, RdLo + Rm * Rs)	S
	16 * 16 bit	SMULxy Rd, Rm, Rs	N Z C*	Rd := Rm[x] * Rs[y]	
		SMULwy Rd, Rm, Rs	N Z C*	Rd := (Rm * Rs[y])[47:16]	
	16 * 16 bit and accumulate	SMLAXy Rd, Rm, Rs, Rn	N Z C*	Rd := Rn + Rm[x] * Rs[y]	Q
		SMLAWy Rd, Rm, Rs, Rn	N Z C*	Rd := Rn + (Rm * Rs[y])[47:16]	Q
	32 * 16 bit and accumulate	SMLALxy RdLo, RdHi, Rm, Rs	N Z C*	RdHi, RdLo := RdHi, RdLo + Rm[x] * Rs[y]	
		SMLAL{X} Rd, Rm, Rs	N Z C*	Rd := Rm[15:0] * RsX[15:0] + Rm[31:16] * RsX[31:16]	Q
	16 * 16 bit and accumulate long	SMULAD{X} Rd, Rm, Rs	N Z C*	Rd := Rn + Rm[15:0] * RsX[15:0] + Rm[31:16] * RsX[31:16]	Q
		SMULAD{X} Rd, Rm, Rs, Rn	N Z C*	RdHi, RdLo := RdHi, RdLo + Rm[15:0] * RsX[15:0] + Rm[31:16] * RsX[31:16]	Q
	Dual signed multiply, add and accumulate	SMULSD{X} Rd, Rm, Rs	N Z C*	Rd := Rm[15:0] * RsX[15:0] - Rm[31:16] * RsX[31:16]	Q
		SMULSD{X} Rd, Rm, Rs, Rn	N Z C*	Rd := Rn + Rm[15:0] * RsX[15:0] - Rm[31:16] * RsX[31:16]	Q
	Signed top word multiply and accumulate	SMULSLD{X} RdLo, RdHi, Rm, Rs	N Z C*	RdHi, RdLo := RdHi, RdLo + Rm[15:0] * RsX[15:0] - Rm[31:16] * RsX[31:16]	
		SMUL{R} Rd, Rm, Rs	N Z C*	Rd := (Rm * Rs)[63:32]	
	Signed word multiply and accumulate with internal 40-bit accumulate	SMMLA{R} Rd, Rm, Rs, Rn	N Z C*	Rd := Rn + (Rm * Rs)[63:32]	
		SMMLS{R} Rd, Rm, Rs, Rn	N Z C*	Rd := Rn - (Rm * Rs)[63:32]	
Divide	Signed or Unsigned Move	MTA Ac, Rm, Rs	N Z C*	Ac := Ac + Rm * Rs	
		MTAPH Ac, Rm, Rs	N Z C*	Ac := Ac + Rm[15:0] * Rs[15:0] + Rm[31:16] * Rs[31:16]	
Move data	NOT top wide	MTAXy Ac, Rm, Rs	N Z C*	Ac := Ac + Rm[x] * Rs[y]	
		RM	N Z C*	Rd := Rn / Rm	
Shift	Arithmetic shift right Logical shift left Logical shift right Rotate right Rotate right with extend	MOV{S} Rd, <Operand2>	N Z C	Rd := Operand2	N
		MVN{S} Rd, <Operand2>	N Z C	Rd := 0xFFFFFFF EOR Operand2	N
Count leading zeros	Compare negative	T2 MOVt Rd, #<imm16>	N Z C	Rd[31:16] := imm16, Rd[15:0] unaffected, imm16 range 0-65535	
		T2 MOV Rd, #<imm16>	N Z C	Rd[15:0] := imm16, Rd[31:16] = 0, imm16 range 0-65535	
Logical	Test Test equivalence AND EOR ORR ORN Bit Clear	XS MRA RdLo, RdHi, Ac	N Z C	RdLo := Ac[31:0], RdHi := Ac[39:32]	
		XS MAR Ac, RdLo, RdHi	N Z C	Ac[31:0] := RdLo, Ac[39:32] := RdHi	
Compare	Test Test equivalence AND EOR ORR ORN Bit Clear	ASR{S} Rd, Rm, <Rs sh>	N Z C	Rd := ASR(Rm, Rslsh)	N
		LSL{S} Rd, Rm, <Rs sh>	N Z C	Rd := LSL(Rm, Rslsh)	N
Logical	Test Test equivalence AND EOR ORR ORN Bit Clear	LSR{S} Rd, Rm, <Rs sh>	N Z C	Rd := LSR(Rm, Rslsh)	N
		ROR{S} Rd, Rm, <Rs sh>	N Z C	Rd := ROR(Rm, Rslsh)	N
Count leading zeros	Compare negative	RRX{S} Rd, Rm	N Z C	Rd := RRX(Rm)	N
		CLZ Rd, Rm	N Z C	Rd := number of leading zeros in Rm	
Logical	Test Test equivalence AND EOR ORR ORN Bit Clear	CMN Rn, <Operand2>	N Z C V	Update CPSR flags on Rn - Operand2	N
		CMN Rn, <Operand2>	N Z C V	Update CPSR flags on Rn + Operand2	N
Logical	Test Test equivalence AND EOR ORR ORN Bit Clear	TST Rn, <Operand2>	N Z C	Update CPSR flags on Rn AND Operand2	N
		TEQ Rn, <Operand2>	N Z C	Update CPSR flags on Rn EOR Operand2	N
Logical	Test Test equivalence AND EOR ORR ORN Bit Clear	AND{S} Rd, Rn, <Operand2>	N Z C	Rd := Rn AND Operand2	N
		EOR{S} Rd, Rn, <Operand2>	N Z C	Rd := Rn EOR Operand2	N
Logical	Test Test equivalence AND EOR ORR ORN Bit Clear	ORR{S} Rd, Rn, <Operand2>	N Z C	Rd := Rn OR Operand2	N
		ORN{S} Rd, Rn, <Operand2>	N Z C	Rd := Rn OR NOT Operand2	T
Logical	Test Test equivalence AND EOR ORR ORN Bit Clear	BIC{S} Rd, Rn, <Operand2>	N Z C	Rd := Rn AND NOT Operand2	N
		BIC{S} Rd, Rn, <Operand2>	N Z C	Rd := Rn AND NOT Operand2	N

ARM and Thumb-2 Instruction Set Quick Reference Card

Operation	§	Assembler	Action	Notes
Bit field	T2	BFC Rd, #<lsb>, #<width>	Rd[(width+lsb-1):lsb] := 0, other bits of Rd unaffected	
	T2	BFI Rd, Rn, #<lsb>, #<width>	Rd[(width+lsb-1):lsb] := Rn[(width-1):0], other bits of Rd unaffected	
	T2	SBFX Rd, Rn, #<lsb>, #<width>	Rd[(width-1):0] := Rn[(width+lsb-1):lsb], Rd[31:width] = Replicate(Rn[(width+lsb-1)])	
	T2	UBFX Rd, Rn, #<lsb>, #<width>	Rd[(width-1):0] := Rn[(width+lsb-1):lsb], Rd[31:width] = Replicate(0)	
Pack	6	PKHBT Rd, Rn, Rm{, LSL #<sh>}	Rd[15:0] := Rn[15:0], Rd[31:16] := (Rm LSL sh)[31:16], sh 0-31.	
	6	PKHTB Rd, Rn, Rm{, ASR #<sh>}	Rd[31:16] := Rn[31:16], Rd[15:0] := (Rm ASR sh)[15:0], sh 1-32.	
Signed extend	6	SXTH Rd, Rm{, ROR #<sh>}	Rd[31:0] := SignExtend((Rm ROR (8 * sh))[15:0]), sh 0-3.	N
	6	SXTB16 Rd, Rm{, ROR #<sh>}	Rd[31:16] := SignExtend((Rm ROR (8 * sh))[23:16]), Rd[15:0] := SignExtend((Rm ROR (8 * sh))[7:0]), sh 0-3.	N
Unsigned extend	6	SXTB Rd, Rm{, ROR #<sh>}	Rd[31:0] := SignExtend((Rm ROR (8 * sh))[7:0]), sh 0-3.	N
	6	UXTH Rd, Rm{, ROR #<sh>}	Rd[31:0] := ZeroExtend((Rm ROR (8 * sh))[15:0]), sh 0-3.	N
Unsigned extend with add	6	UXTB16 Rd, Rm{, ROR #<sh>}	Rd[31:16] := ZeroExtend((Rm ROR (8 * sh))[23:16]), Rd[15:0] := ZeroExtend((Rm ROR (8 * sh))[7:0]), sh 0-3.	N
	6	UXTB Rd, Rm{, ROR #<sh>}	Rd[31:0] := ZeroExtend((Rm ROR (8 * sh))[7:0]), sh 0-3.	N
Signed extend with add	6	SXTAH Rd, Rn, Rm{, ROR #<sh>}	Rd[31:0] := Rn[31:0] + SignExtend((Rm ROR (8 * sh))[15:0]), sh 0-3.	
	6	SXTAB16 Rd, Rn, Rm{, ROR #<sh>}	Rd[31:16] := Rn[31:16] + SignExtend((Rm ROR (8 * sh))[23:16]), Rd[15:0] := Rn[15:0] + SignExtend((Rm ROR (8 * sh))[7:0]), sh 0-3.	
Unsigned extend with add	6	SXTAB Rd, Rn, Rm{, ROR #<sh>}	Rd[31:0] := Rn[31:0] + SignExtend((Rm ROR (8 * sh))[7:0]), sh 0-3.	
	6	UXTAH Rd, Rn, Rm{, ROR #<sh>}	Rd[31:16] := Rn[31:16] + ZeroExtend((Rm ROR (8 * sh))[23:16]), Rd[15:0] := Rn[15:0] + ZeroExtend((Rm ROR (8 * sh))[7:0]), sh 0-3.	
Reverse	6	UXTAB Rd, Rn, Rm{, ROR #<sh>}	Rd[31:0] := Rn[31:0] + ZeroExtend((Rm ROR (8 * sh))[7:0]), sh 0-3.	
	T2	RBIT Rd, Rm	For (i = 0; i < 32; i++) : Rd[i] = Rm[31-i]	
Select	6	REV Rd, Rm	Rd[31:24] := Rm[7:0], Rd[23:16] := Rm[15:8], Rd[15:8] := Rm[23:16], Rd[7:0] := Rm[31:24]	N
	6	REV16 Rd, Rm	Rd[15:8] := Rm[7:0], Rd[7:0] := Rm[15:8], Rd[31:24] := Rm[23:16], Rd[23:16] := Rm[31:24]	N
If-Then	6	REVSH Rd, Rm	Rd[15:8] := Rm[7:0], Rd[7:0] := Rm[15:8], Rd[31:16] := Rm[7] * &FFFF	N
	6	SEL Rd, Rn, Rm	Rd[7:0] := Rn[7:0] if GE[0] = 1, else Rd[7:0] := Rm[7:0] Bits[15:8], [23:16], [31:24] selected similarly by GE[1], GE[2], GE[3]	
Branch	T2	IT{pattern} {cond}	Makes up to four following instructions conditional, according to pattern. pattern is a string of up to three letters. Each letter can be T (Then) or E (Else).	T U
			The first instruction after IT has condition cond. The following instructions have condition cond if the corresponding letter is T, or the inverse of cond if the corresponding letter is E. See Table Condition Field for available condition codes.	
Move to or from PSR	4T	BL <label>	PC := label, label is this instruction ±32MB (T2: ±16MB; T: -252 - +256B)	N, B
	5T	BLX <label>	LR := address of next instruction, PC := label, label is this instruction ±32MB (T2: ±16MB). PC := Rm. Target is Thumb if Rm[0] is 1, ARM if Rm[0] is 0. LR := address of next instruction, PC := label, Change instruction set. label is this instruction ±32MB (T2: ±16MB).	N C
Processor state change	5	BLX Rm	LR := address of next instruction, PC := Rm[31:1]. Change to Thumb if Rm[0] is 1, to ARM if Rm[0] is 0.	N
	5J	BXJ Rm	Change to Jazelle state if available	N T U
Processor state change	T2	CB{N}Z Rn, <label>	If Rn {== or !=} 0 then PC := label, label is (this instruction + 4-130).	T U
	T2	TBB {Rn, Rm}	PC = PC + ZeroExtend(Memory(Rn + Rm, 1) < 1). Branch range 4-512. Rn can be PC.	T U
Processor state change	T2	TBB {Rn, Rm, LSL #1}	PC = PC + ZeroExtend(Memory(Rn + Rm << 1, 2) << 1). Branch range 4-131072. Rn can be PC.	T U
		MRS Rd, <PSR>	Rd := PSR	
Processor state change		MSR <PSR>_<fields>, Rm	PSR := Rm (selected bytes only)	
		MSR <PSR>_<fields>, #<imm8>	PSR := imm8 (selected bytes only)	
Processor state change	6	CPSID <flags> {, #<p_mode>}	Disable specified interrupts, optional change mode.	U, N
	6	CPSIE <flags> {, #<p_mode>}	Enable specified interrupts, optional change mode.	U, N
Processor state change	6	CPS #<p_mode>	U	
	6	SETEND <endianness>	Sets endianness for loads and saves. <endianness> can be BE (Big Endian) or LE (Little Endian).	U, N

ARM Instruction Set Quick Reference Card

Single data item loads and stores		§	Assembler	Action if <op> is LDR	Action if <op> is STR	Notes
Load or store word, byte or halfword	Immediate offset		<op> {size}{T} Rd, [Rn, #<offset>}] {!}	Rd := [address, size]	[address, size] := Rd	1, N
	Post-indexed, immediate		<op> {size}{T} Rd, [Rn], #<offset>	Rd := [address, size]	[address, size] := Rd	2
	Register offset		<op> {size} Rd, [Rn, +/-Rm {, <opsh>}] {!}	Rd := [address, size]	[address, size] := Rd	3, N
	Post-indexed, register		<op> {size}{T} Rd, [Rn], +/-Rm {, <opsh>}	Rd := [address, size]	[address, size] := Rd	4
Load or store doubleword	PC-relative		<op> {size} Rd, <label>	Rd := [label, size]	Not available	5, N
	Immediate offset	5E	<op>D Rd1, Rd2, [Rn {, #<offset>}] {!}	Rd1 := [address], Rd2 := [address + 4]	[address] := Rd1, [address + 4] := Rd2	6, 9
	Post-indexed, immediate	5E	<op>D Rd1, Rd2, [Rn], #<offset>	Rd1 := [address], Rd2 := [address + 4]	[address] := Rd1, [address + 4] := Rd2	6, 9
	Register offset	5E	<op>D Rd1, Rd2, [Rn, +/-Rm {, <opsh>}] {!}	Rd1 := [address], Rd2 := [address + 4]	[address] := Rd1, [address + 4] := Rd2	7, 9
	Post-indexed, register	5E	<op>D Rd1, Rd2, [Rn], +/-Rm {, <opsh>}	Rd1 := [address], Rd2 := [address + 4]	[address] := Rd1, [address + 4] := Rd2	7, 9
	PC-relative	5E	<op>D Rd1, Rd2, <label>	Rd1 := [label], Rd2 := [label + 4]	Not available	8, 9

Preload data or instruction		§ (PLD)	§ (PLI)	Assembler	Action if <op> is PLD	Action if <op> is PLI	Notes
	Immediate offset	5E		<op> [Rn {, #<offset>}]	Preload [address, 32] (data)	Preload [address, 32] (instruction)	1, C
	Register offset	5E		<op> [Rn, +/-Rm {, <opsh>}]	Preload [address, 32] (data)	Preload [address, 32] (instruction)	3, C
	PC-relative	5E		<op> <label>	Preload [label, 32] (data)	Preload [label, 32] (instruction)	5, C

Other memory operations		§	Assembler	Action		Notes
Load multiple	Block data load		LDM{IA IB DA DB} Rn{!}, <reglist>-PC>	Load list of registers from [Rn]		N, I
	return (and exchange)		LDM{IA IB DA DB} Rn{!}, <reglist>-PC>	Load registers, PC := [address][31:1] (§ 5T: Change to Thumb if [address][0] is 1)		I
	and restore CPSR		LDM{IA IB DA DB} Rn{!}, <reglist>-PC>^	Load registers, branch (§ 5T: and exchange), CPSR := SPSR, Exception modes only.		I
	User mode registers		LDM{IA IB DA DB} Rn, <reglist>-PC>^	Load list of User mode registers from [Rn]. Privileged modes only.		I
Pop			POP <reglist>	Canonical form of LDM SP!, <reglist>		N
Load exclusive	Semaphore operation	6	LDREX Rd, [Rn]	Rd := [Rn], tag address as exclusive access. Outstanding tag set if not shared address.		
	Halfword or Byte	6K	LDREX(H B) Rd, [Rn]	Rd[15:0] := [Rn] or Rd[7:0] := [Rn], tag address as exclusive access.		
	Doubleword	6K	LDREXD Rd1, Rd2, [Rn]	Outstanding tag set if not shared address. Rd, Rn not PC.		9
Store multiple	Push, or Block data store		STM{IA IB DA DB} Rn{!}, <reglist>	Store list of registers to [Rn]		N, I
	User mode registers		STM{IA IB DA DB} Rn{!}, <reglist>^	Store list of User mode registers to [Rn]. Privileged modes only.		I
			PUSH <reglist>	Canonical form of STMDB SP!, <reglist>		N
Push	Semaphore operation	6	STREX Rd, Rm, [Rn]	If allowed, [Rn] := Rm, clear exclusive tag. Rd := 0. Else Rd := 1. Rd, Rm, Rn not PC.		
	Halfword or Byte	6K	STREX(H B) Rd, Rm, [Rn]	If allowed, [Rn] := Rm[15:0] or [Rn] := Rm[7:0], clear exclusive tag. Rd := 0. Else Rd := 1.		
	Doubleword	6K	STREXD Rd, Rm1, Rm2, [Rn]	If allowed, [Rn] := Rm1, [Rn+4] := Rm2, clear exclusive tags, Rd := 0. Else Rd := 1.		9
Clear exclusive		6K	CLEEX	Clear local processor exclusive tag		C

Notes: availability and range of options for Load, Store, and Preload operations

Note	ARM Word, B, D	ARM SB, H, SH	ARM T, BT	Thumb-2 Word, B, SB, H, SH, D	Thumb-2 T, BT, SBT, HT, SHT
1	offset: -4095 to +4095	offset: -255 to +255	Not available	offset: -255 to +255 if writeback, -255 to +4095 otherwise	offset: 0 to +255, writeback not allowed
2	offset: -4095 to +4095	offset: -255 to +255	offset: -4095 to +4095	offset: -255 to +255	Not available
3	Full range of {, <opsh>}	{, <opsh>} not allowed	Not available	<opsh> restricted to LSL #<sh>, <sh> range 0 to 3	Not available
4	Full range of {, <opsh>}	{, <opsh>} not allowed	Full range of {, <opsh>}	Not available	Not available
5	label within +/- 4092 of current instruction	Not available	Not available	label within +/- 4092 of current instruction	Not available
6	offset: -255 to +255	-	-	offset: -1020 to +1020, must be multiple of 4.	-
7	{, <opsh>} not allowed	-	-	Not available	-
8	label within +/- 252 of current instruction	-	-	Not available	-
9	Rd1 even, and not r14, Rd2 == Rd1 + 1.	-	-	Rd1 := PC, Rd2 := PC	-

ARM Instruction Set Quick Reference Card

Coprocessor operations	§	Assembler	Action	Notes
Data operations		CDP{2} <copr>, <op1>, CRd, CRn, CRm{, <op2>}	Coprocessor defined	C2
Move to ARM register from coprocessor		MRC{2} <copr>, <op1>, Rd, CRn, CRm{, <op2>}	Coprocessor defined	C2
Two ARM register move	5E	MRRC <copr>, <op1>, Rd, Rn, CRm	Coprocessor defined	C
Alternative two ARM register move	6	MRRC2 <copr>, <op1>, Rd, Rn, CRm	Coprocessor defined	C2
Move to coproc from ARM reg		MCR{2} <copr>, <op1>, Rd, CRn, CRm{, <op2>}	Coprocessor defined	C
Two ARM register move	5E	MCCR <copr>, <op1>, Rd, Rn, CRm	Coprocessor defined	C
Alternative two ARM register move	6	MCCR2 <copr>, <op1>, Rd, Rn, CRm	Coprocessor defined	C2
Loads and stores, pre-indexed		<op>{2} <copr>, CRd, [Rn, #+/-offset8*4] {, !}	op: LDC or STC. offset: multiple of 4 in range 0 to 1020.	C2
Loads and stores, zero offset		<op>{2} <copr>, CRd, [Rn] {, 8-bit copro. option}	op: LDC or STC.	C2
Loads and stores, post-indexed		<op>{2} <copr>, CRd, [Rn], #+/-offset8*4>	op: LDC or STC. offset: multiple of 4 in range 0 to 1020.	C2

Miscellaneous operations	§	Assembler	Action	Notes
Swap word		SWP Rd, Rm, [Rn]	temp := [Rn], [Rn] := Rm, Rd := temp.	D
Swap byte		SWPB Rd, Rm, [Rn]	temp := ZeroExtend([Rn][7:0]), [Rn][7:0] := Rm[7:0], Rd := temp	D
Store return state	6	SRS{IA IB DA DB} SP{!}, #<p_mode>	[SPm] := LR, [SPm + 4] := CPSR	C, I
Return from exception	6	RFE{IA IB DA DB} Rn{!}	PC := [Rn], CPSR := [Rn + 4]	C, I
Breakpoint	5	BKPT <imm16>	Prefetch abort or enter debug state. 16-bit bitfield encoded in instruction.	C, N
Secure Monitor Call	Z	SMC <imm16>	Secure Monitor Call exception. 16-bit bitfield encoded in instruction. Formerly SMI.	N
Supervisor Call		SVC <imm24>	Supervisor Call exception. 24-bit bitfield encoded in instruction. Formerly SWI.	N
No operation	6	NOP	None, might not even consume any time.	N
Hints				
Debug Hint	7	DBG	Provide hint to debug and related systems.	C
Data Memory Barrier	7	DMB	Ensure the order of observation of memory accesses.	C
Data Synchronization Barrier	7	DSB	Ensure the completion of memory accesses.	C
Instruction Synchronization Barrier	7	ISB	Flush processor pipeline and branch prediction logic.	C
Set event	T2	SEV	Signal event in multiprocessor system. NOP if not implemented.	N
Wait for event	T2	WFE	Wait for event, IRQ, FIQ, Imprecise abort, or Debug entry request. NOP if not implemented.	N
Wait for interrupt	T2	WFI	Wait for IRQ, FIQ, Imprecise abort, or Debug entry request. NOP if not implemented.	N
Yield	T2	YIELD	Yield control to alternative thread. NOP if not implemented.	N

Notes	N	P	Q	R	S	T	U
A Not available in Thumb state.	Some or all forms of this instruction are 16-bit (Narrow) instructions in Thumb-2 code. For details see the <i>Thumb 16-bit Instruction Set (UAL) Quick Reference Card</i> .						
B Can be conditional in Thumb state without having to be in an IT block.	Rn can be the PC in Thumb state in this instruction.						
C Condition codes are not allowed in ARM state.	Sets the Q flag if saturation (addition or subtraction) or overflow (multiplication) occurs. Read and reset the Q flag using MRS and MSR.						
C2 The optional 2 is available from ARMv5. It provides an alternative operation. Condition codes are not allowed for the alternative form in ARM state.	<sh> range is 1-32 in the ARM instruction.						
D Deprecated. Use LDREX and STREX instead.	The S modifier is not available in the Thumb-2 instruction.						
G Updates the four GE flags in the CPSR based on the results of the individual operations.	Not available in ARM state.						
I IA is the default, and is normally omitted.	Not allowed in an IT block. Condition codes not allowed in either ARM or Thumb state.						
L ARM: <imm8m>. 16-bit Thumb: multiple of 4 in range 0-1020. 32-bit Thumb: 0-4095.							

ARM Instruction Set
Quick Reference Card

ARM architecture versions	
<i>n</i>	ARM architecture version <i>n</i> and above
<i>n</i> T, <i>n</i> J	T or J variants of ARM architecture version <i>n</i> and above
5E	ARM v5E, and 6 and above
T2	All Thumb-2 versions of ARM v6 and above
6K	ARMv6K and above for ARM instructions, ARMv7 for Thumb
Z	All Security extension versions of ARMv6 and above
RM	ARMv7-R and ARMv7-M only
XS	XScale coprocessor instruction

Flexible Operand 2	
Immediate value	#<imm8m>
Register, optionally shifted by constant (see below)	Rm {, <opsh> }
Register, logical shift left by register	Rm, LSL Rs
Register, logical shift right by register	Rm, LSR Rs
Register, arithmetic shift right by register	Rm, ASR Rs
Register, rotate right by register	Rm, ROR Rs

Register, optionally shifted by constant	
(No shift)	Rm
Logical shift left	Rm, LSL #<shift>
Logical shift right	Rm, LSR #<shift>
Arithmetic shift right	Rm, ASR #<shift>
Rotate right	Rm, ROR #<shift>
Rotate right with extend	Rm, RRX
	Same as Rm, LSL #0
	Allowed shifts 0-31
	Allowed shifts 1-32
	Allowed shifts 1-32
	Allowed shifts 1-31

PSR fields	
Suffix	Meaning
C	Control field mask byte
F	Flags field mask byte
S	Status field mask byte
X	Extension field mask byte

Proprietary Notice

Words and logos marked with ® or ™ are registered trademarks or trademarks owned by ARM Limited. Other brands and names mentioned herein may be the trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM in good faith. However, all warranties implied or expressed, including but not limited to implied warranties of merchantability, or fitness for purpose, are excluded.

This reference card is intended only to assist the reader in the use of the product. ARM Ltd shall not be liable for any loss or damage arising from the use of any information in this reference card, or any error or omission in such information, or any incorrect use of the product.

Condition Field	
Mnemonic	Description
EQ	Equal
NE	Not equal
CS / HS	Carry Set / Unsigned higher or same
CC / LO	Carry Clear / Unsigned lower
MI	Negative
PL	Positive or zero
VS	Overflow
VC	No overflow
HI	Unsigned higher
LS	Unsigned lower or same
GE	Signed greater than or equal
LT	Signed less than
GT	Signed greater than
LE	Signed less than or equal
AL	Always (normally omitted)
All ARM instructions (except those with Note C or Note U) can have any one of these condition codes after the instruction mnemonic (that is, before the first space in the instruction as shown on this card). This condition is encoded in the instruction.	
All Thumb-2 instructions (except those with Note U) can have any one of these condition codes after the instruction mnemonic. This condition is encoded in a preceding IT instruction (except in the case of conditional Branch instructions). Condition codes in instructions must match those in the preceding IT instruction.	
On processors without Thumb-2, the only Thumb instruction that can have a condition code is B <label>.	

Processor Modes	
16	User
17	FIQ Fast Interrupt
18	IRQ Interrupt
19	Supervisor
23	Abort
27	Undefined
31	System

Prefixes for Parallel Instructions	
S	Signed arithmetic modulo 2 ⁸ or 2 ¹⁶ , sets CPSR GE bits
Q	Signed saturating arithmetic
SH	Signed arithmetic, halving results
U	Unsigned arithmetic modulo 2 ⁸ or 2 ¹⁶ , sets CPSR GE bits
UQ	Unsigned saturating arithmetic
UH	Unsigned arithmetic, halving results

Document Number

ARM QRC 0001L

Change Log

Issue	Date	Change	Issue	Date	Change
A	June 1995	First Release	B	Sept 1996	Second Release
C	Nov 1998	Third Release	D	Oct 1999	Fourth Release
E	Oct 2000	Fifth Release	F	Sept 2001	Sixth Release
G	Jan 2003	Seventh Release	H	Oct 2003	Eighth Release
I	Dec 2004	Ninth Release	J	May 2005	RVCT 2.2 SP1
K	March 2006	RVCT 3.0	L	March 2007	RVCT 3.1